AGENT-ORCHESTRATED AUTOMATED QUESTION PAPER VALIDATION USING MCP TOOLS AND RAG

A PROJECT REPORT

Submitted by

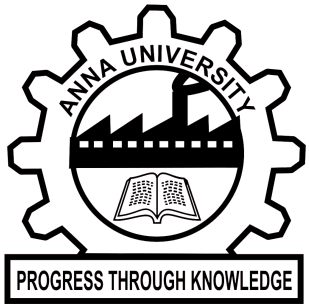
JANANI SRI S K
(2024178024)

Submitted to the Faculty of

INFORMATION AND COMMUNICATION ENGINEERING

in partial fulfillment for the award of the degree of

MASTER OF COMPUTER APPLICATIONS



DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY
COLLEGE OF ENGINEERING, GUINDY
ANNA UNIVERSITY
CHENNAI – 600 025

MAY 2026

ANNA UNIVERSITY

CHENNAI – 600 025

BONA FIDE CERTIFICATE

Certified that this project report titled **AGENT-ORCHESTRATED AUTOMATED QUESTION PAPER VALIDATION USING MCP TOOLS AND RAG** is the bona fide work of **JANANI SRI S K (2024178024)** who carried out the project work under my supervision. Certified further that to the best of my knowledge and belief, the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or an award was conferred on an earlier occasion on this or any other candidate.

PLACE: CHENNAI
DATE: 06.04.2026

Dr. K. KULOTHUNGAN
ASSOCIATE PROFESSOR
PROJECT GUIDE
DEPARTMENT OF IST, CEG
ANNA UNIVERSITY
CHENNAI – 600 025

COUNTERSIGNED

Dr. P. YOGESH
HEAD OF THE DEPARTMENT
DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY
COLLEGE OF ENGINEERING, GUINDY
ANNA UNIVERSITY
CHENNAI – 600 025

ABSTRACT

The increasing adoption of Outcome-Based Education (OBE) frameworks in higher education has introduced stringent requirements for examination question paper design, demanding precise alignment with syllabus boundaries, Course Outcomes (COs), and Bloom's Taxonomy cognitive levels. Traditional manual validation workflows are time-consuming, subjective, and prone to inconsistency, often failing to detect out-of-syllabus questions, imbalanced cognitive distributions, and incomplete Course Outcome coverage before examination review.

Recent advances in Natural Language Processing, transformer-based language models, and Retrieval-Augmented Generation (RAG) have created new opportunities for automating academic validation tasks. However, unconstrained generative systems introduce hallucination, lack of syllabus grounding, and insufficient explainability, rendering them unsuitable for high-stakes institutional assessment environments.

To address these limitations, this project proposes an Agent-Orchestrated Automated Question Paper Validation System that integrates semantic retrieval, transformer-based classification, pipeline evaluation, and adaptive question generation within a deterministic, syllabus-grounded architecture. The system follows the Model Context Protocol (MCP) tool-use pattern, where a central AgentOrchestrator invokes modular, context-aware tools — comprising a Retrieval Tool, Classification Tool, Matching Tool, Generation Tool, Evaluation Tool, and Export Tool — coordinating their execution based on user intent and domain context. The system constructs a structured academic knowledge base from official syllabus documents and reference textbooks, generating dense vector embeddings using the Sentence-BERT model `all-MiniLM-L6-v2` and indexing them in course-isolated FAISS indices stored within MongoDB GridFS. A hybrid two-stage retrieval mechanism, combining bi-encoder filtering with cross-encoder reranking (`ms-marco-MiniLM-L-6-v2`), performs fine-grained topic matching and Course Outcome mapping for each submitted question. Bloom's Taxonomy classification is performed using a fine-tuned DeBERTa-v3-base model achieving approximately 82% accuracy across six cognitive levels (BT1–BT6). Difficulty and strength scores are computed deterministically from Bloom-level weights and semantic heuristics, and all results are persisted in structured JSON format with PDF and DOCX export support.

Beyond per-question evaluation, the system incorporates a pipeline validation module that benchmarks end-to-end accuracy against labelled ground-truth datasets using strict exact, ± 1 relaxed Bloom matching, semantic cosine topic matching, and exact CO matching strategies. An adaptive question generation and gap analysis module further analyses

evaluated question papers to detect Bloom-level imbalances, missing or underrepresented Course Outcomes, and uncovered syllabus units. Targeted replacement questions are automatically generated using the Groq LLM (`llama-3.3-70b-versatile`) with syllabus-grounded prompts retrieved via FAISS and cross-encoder reranking, or guided interactively for Bloom-level improvements. The complete pipeline is orchestrated through a FastAPI-based central agent that enforces deterministic execution order across all modules.

The proposed system demonstrates a transparent, explainable, and reproducible approach to automated academic assessment, addressing key limitations of existing solutions in syllabus compliance, cognitive-level accuracy, and adaptive quality improvement of examination question papers. **Keywords:** Outcome-Based Education, Bloom's Taxonomy,

Retrieval-Augmented Generation, FAISS, Transformer Models, Question Paper Validation, Adaptive Question Generation, Gap Analysis, DeBERTa-v3-base, Cross-Encoder Reranking, Model Context Protocol, Agent Orchestration.

Table of Contents

List of Figures

List of Abbreviations

OBE	Outcome-Based Education
CO	Course Outcome
NLP	Natural Language Processing
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
BERT	Bidirectional Encoder Representations from Transformers
DeBERTa	Decoding-enhanced BERT with Disentangled Attention
SBERT	Sentence-BERT
FAISS	Facebook AI Similarity Search
GridFS	Grid File System
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
PDF	Portable Document Format
DOCX	Microsoft Word Open XML Document
CLI	Command Line Interface
UI	User Interface
BT	Bloom's Taxonomy Level
AdamW	Adaptive Moment Estimation with Weight Decay
RLHF	Reinforcement Learning from Human Feedback
LMS	Learning Management System
NFKC	Normalization Form Compatibility Composition
SVM	Support Vector Machine
F1	F1-Score

CHAPTER 1

INTRODUCTION

The increasing adoption of Outcome-Based Education (OBE) frameworks in higher education has significantly transformed the design, structure, and evaluation of academic assessments. Under OBE, institutions are required to ensure that examination question papers are not only aligned with the prescribed syllabus but are also accurately mapped to Course Outcomes (COs) and distributed appropriately across Bloom's Taxonomy cognitive levels. In addition to conceptual alignment, question papers must maintain balanced difficulty levels and provide adequate coverage of all instructional units to ensure fairness and academic rigor.

Traditionally, the preparation and validation of question papers are performed manually by subject experts. Although experienced faculty members are capable of maintaining academic standards, the process remains time-consuming, subjective, and susceptible to inconsistency. Manual validation lacks standardization and reproducibility, particularly in large-scale examinations that involve multiple sections, diverse cognitive levels, and complex syllabus structures. Issues such as inclusion of out-of-syllabus questions, uneven CO coverage, improper Bloom-level distribution, and ambiguously framed questions may remain undetected until after examination review. Such inconsistencies highlight the limitations of purely manual assessment workflows in modern academic environments.

Recent advancements in Natural Language Processing (NLP) and transformer-based language models have enabled systems to perform deep semantic analysis of textual content. Techniques such as sentence embeddings, contextual similarity search, and Retrieval-Augmented Generation (RAG) offer the potential to model academic content in a structured and machine-interpretable manner. These developments provide opportunities for automating academic validation tasks that previously required extensive manual effort. However, the direct application of large language models (LLMs) in academic assessment introduces new challenges. These include hallucination, lack of strict syllabus grounding, insufficient explainability, and unpredictability in output generation. Such limitations make unconstrained generative systems unsuitable for high-stakes educational settings where transparency, accountability, and determinism are essential.

To address these challenges, this project proposes an **Agent-Orchestrated Automated Question Paper Validation System** that integrates semantic retrieval, transformer-based classification, and syllabus-grounded generation mechanisms within a deterministic framework. The system is designed to ensure transparent, consistent, and review-defensible evaluation of academic questions while supporting both validation and controlled question generation.

1.1 PROBLEM STATEMENT

Designing examination question papers under OBE guidelines involves satisfying multiple academic constraints simultaneously. Each question must strictly adhere to syllabus boundaries, correspond to appropriate Course Outcomes, align with a defined Bloom's Taxonomy level, and contribute to an overall balanced cognitive and difficulty distribution within the question paper. Ensuring compliance with these requirements through manual validation is both labor-intensive and error-prone.

In existing academic workflows, validation tasks are largely performed through human judgment. As a result, syllabus compliance is not always rigorously enforced, Course Outcome mapping may vary across evaluators, and Bloom-level distribution may become imbalanced. Difficulty estimation is often subjective and lacks quantifiable justification. Furthermore, inconsistencies may arise between individual question evaluation and batch evaluation of full question papers, particularly when preprocessing or formatting differences are involved. Another major limitation is the absence of explainable decision mechanisms that justify why a question is considered valid, out-of-syllabus, or assigned a particular cognitive level.

At the same time, the emergence of generative AI tools has introduced new possibilities for automated question generation. However, when such systems are used without structured grounding, they may produce questions beyond the prescribed syllabus or misaligned with defined academic objectives. In institutional contexts, uncontrolled generation is unsuitable because examination systems require traceability, reproducibility, and strict compliance with curricular standards. Furthermore, existing systems lack a structured tool-use orchestration pattern, where a central agent can invoke modular, context-aware tools for retrieval, classification, matching, generation, and evaluation in a coordinated and deterministic sequence.

Beyond individual question validation, existing systems lack mechanisms to benchmark their own end-to-end prediction accuracy against labelled ground-truth datasets, making it difficult to assess pipeline reliability or identify systematic errors across Bloom levels, topics, and Course Outcomes. Additionally, the absence of automated coverage gap detection means that structural deficiencies in question papers — such as Bloom-level imbalances, underrepresented or absent Course Outcomes, and uncovered syllabus units — often go undetected until after manual review, with no mechanism to guide or automate targeted correction.

Therefore, there exists a need for an intelligent, syllabus-grounded, explainable, and deterministic system capable of validating and generating academic questions while ensuring full compliance with Outcome-Based Education standards.

1.2 OBJECTIVES

The primary objective of this project is to develop an automated, agent-orchestrated system for question paper validation and controlled question generation. The system aims to construct a structured academic knowledge base from syllabus documents and reference textbooks, enabling machine-interpretable representation of instructional content. It seeks to implement a multi-stage semantic validation pipeline that ensures strict adherence to syllabus boundaries before classification or scoring is performed.

Another core objective is to map validated questions to appropriate Course Outcomes using contextual similarity rather than surface-level keyword matching. The system is designed to classify questions into Bloom's Taxonomy levels ranging from BT1 to BT6 using a fine-tuned DeBERTa-v3-base model trained for multi-class cognitive prediction. In addition, the system aims to compute difficulty and strength scores based on cognitive demand, semantic complexity, and model confidence metrics, thereby introducing quantifiable evaluation measures.

The project further aims to ensure consistency between single-question evaluation and batch PDF-based evaluation through a unified preprocessing pipeline. The system is additionally designed to follow the Model Context Protocol tool-use pattern, implementing modular tools — Retrieval, Classification, Matching, Generation, Evaluation, and Export — invoked by a central AgentOrchestrator that coordinates execution based on user intent and domain context. The system also incorporates a pipeline validation module that benchmarks end-to-end prediction accuracy against labelled ground-truth datasets using relaxed Bloom matching, semantic cosine topic matching, and exact CO matching strategies, enabling iterative improvement of pipeline components. Additionally, an adaptive gap analysis module is included to automatically detect Bloom-level imbalances, missing or underrepresented Course Outcomes, and uncovered syllabus units across evaluated question papers, triggering targeted question generation or guided faculty intervention. Finally, the system incorporates Retrieval-Augmented Generation to enable syllabus-constrained question generation, where generated questions are re-evaluated through the same validation pipeline to maintain academic integrity and compliance.

1.3 CHALLENGES

Automating question paper validation under OBE guidelines presents several challenges in real-world academic environments. Syllabus boundaries in natural language are inherently ambiguous, and questions may exhibit surface-level similarity to multiple topics across different instructional units. Distinguishing genuine syllabus alignment from superficial lexical overlap requires robust semantic validation mechanisms beyond simple keyword matching.

The lack of standardized labeled datasets for Bloom-level classification across diverse academic domains complicates model training and generalization. Additionally, ensuring consistent evaluation behavior between single-question and batch-PDF workflows demands a unified preprocessing pipeline that handles diverse formatting patterns, numbering conventions, and structural variations in examination documents.

Scalability is another challenge, as institutions manage multiple courses with distinct syllabi, Course Outcomes, and reference materials. Preventing cross-domain semantic interference while maintaining independent evaluation contexts for each course requires careful architectural design. Furthermore, integrating controlled generation within the same deterministic validation framework introduces the challenge of ensuring that generated questions fully comply with syllabus boundaries, cognitive requirements, and difficulty constraints.

Benchmarking end-to-end pipeline accuracy against ground-truth annotations introduces additional complexity, as topic matching must account for semantic variation in topic phrasing rather than relying on exact string comparison. Detecting coverage gaps across multiple dimensions simultaneously — Bloom levels, Course Outcomes, and syllabus units — while mapping gaps to actionable generation targets without hardcoded positional assumptions further adds to the architectural complexity of the system. Designing a structured MCP-style tool-use orchestration layer that maintains deterministic execution order while allowing modular, independently testable tools introduces further architectural and engineering challenges.

1.4 MOTIVATION

The increasing adoption of OBE frameworks in higher education institutions creates a pressing need for automated, transparent, and reliable question paper validation mechanisms. Manual evaluation processes are inherently inconsistent, time-intensive, and difficult to scale across large institutions managing multiple courses and examination cycles.

The availability of advanced transformer-based language models and efficient vector retrieval systems provides an opportunity to address these limitations through semantic grounding and deterministic orchestration. Early automation of validation and generation tasks can significantly reduce faculty workload while improving compliance with academic standards.

This project is motivated by the objective of designing a scalable, interpretable, and effective academic assessment framework that bridges the gap between theoretical NLP capabilities and practical institutional examination workflows. By combining semantic embeddings, hybrid validation, Bloom classification, and syllabus-constrained generation within a unified system, the proposed work aims to deliver measurable improvements in

question paper quality and OBE compliance.

1.5 PROPOSED WORK

This project proposes an **Agent-Orchestrated Automated Question Paper Validation and Generation System** that integrates semantic retrieval, transformer-based classification, and controlled generation within a deterministic, syllabus-grounded architecture.

The system constructs a structured academic knowledge base from official syllabus documents and reference textbooks. Syllabus content is normalized into instructional units and atomic subtopics, with explicit Course Outcome mappings. Textbook content is chunked and embedded using Sentence-BERT to generate dense vector representations indexed within course-isolated FAISS structures. The system follows the Model Context Protocol tool-use pattern, where a central AgentOrchestrator receives user intent and coordinates the invocation of modular, context-aware tools — a Retrieval Tool built on FAISS and cross-encoder reranking, a Classification Tool using the fine-tuned DeBERTa-v3-base model, a Matching Tool for semantic CO and topic alignment, a Generation Tool powered by the Groq LLM, an Evaluation Tool for pipeline metrics computation, and an Export Tool for PDF and DOCX report generation.

A multi-stage validation pipeline processes submitted questions through domain-level gating, FAISS-based retrieval, bi-encoder filtering, and cross-encoder reranking to verify syllabus compliance. Validated questions are subsequently classified into Bloom's Taxonomy levels using a fine-tuned DeBERTa-v3-base model, and Course Outcome mapping is performed through semantic similarity comparison.

Difficulty and strength scores are computed deterministically based on cognitive level, semantic complexity, and heuristic indicators. The framework supports both single-question evaluation and batch PDF-based evaluation through a unified preprocessing mechanism. A pipeline validation module benchmarks end-to-end accuracy against labelled ground-truth datasets using complementary matching strategies — relaxed Bloom matching, semantic cosine topic matching, and exact CO matching — providing both a user-facing metrics dashboard and a developer-level CLI diagnostic tool for iterative pipeline improvement. An adaptive gap analysis module further analyses evaluated question papers to detect Bloom-level imbalances, missing or underrepresented Course Outcomes, and uncovered syllabus units, and either presents guided faculty correction workflows or triggers automated syllabus-grounded question generation via the Groq LLM with FAISS-retrieved and cross-encoder-reranked context. Additionally, a Retrieval-Augmented Generation module enables controlled question generation constrained by syllabus units, Course Outcomes, and Bloom-level requirements, with generated questions re-evaluated through the same validation pipeline.

1.6 ORGANIZATION OF THE REPORT

The report is systematically organized into chapters to provide a clear and structured presentation of the project. Each chapter focuses on a specific aspect of the work, beginning with the background and problem definition, followed by system design, implementation, evaluation, and concluding with the results and future scope. The organisation of the report is outlined as follows:

Chapter 1 presents an introduction to the Agent-Orchestrated Automated Question Paper Validation System, outlining the problem statement, project objectives, proposed solution overview, and the organization of the report.

Chapter 2 presents a comprehensive review of literature related to automated academic assessment systems, transformer-based semantic models, Bloom's Taxonomy classification techniques, Retrieval-Augmented Generation frameworks, and agent orchestration patterns. It analyzes previously published research works, highlighting their methodologies, findings, and limitations, and identifies research gaps addressed by the proposed system.

Chapter 3 describes the overall architecture of the proposed Agent-Orchestrated Question Paper Validation and Generation System. This chapter explains the system model, knowledge base construction process, embedding mechanisms, hybrid retrieval design, and the agent orchestration pipeline in detail.

Chapter 4 discusses the implementation details of the proposed system. It includes the dataset description, preprocessing steps, module descriptions for knowledge setup, topic matching, Bloom classification, scoring, pipeline validation, and RAG-based adaptive question generation, along with the agent orchestration design.

Chapter 5 presents implementation snapshots of the running system. This chapter demonstrates the runtime behaviour of all seven pipeline modules through representative screenshots of the React-based frontend and the CLI diagnostic tool, covering knowledge base construction, single-question and batch evaluation, evaluation history, pipeline metrics computation, and adaptive gap analysis with question generation.

Chapter 6 focuses on the quantitative performance evaluation of the proposed system. This chapter presents Bloom classification training metrics, cross-validation results, end-to-end pipeline accuracy on a labelled dataset, topic similarity statistics, Bloom confusion patterns, and Course Outcome mismatch analysis.

Chapter 7 concludes the report by summarizing the key contributions of the project

across all seven pipeline modules and discussing possible future enhancements and research directions related to agent-orchestrated academic assessment systems.

CHAPTER 2

LITERATURE SURVEY

This chapter presents a comprehensive review of existing research related to automated academic assessment, question classification, semantic validation, and Retrieval-Augmented Generation (RAG). The evolution of educational assessment systems reflects a gradual transition from rule-based mechanisms to advanced transformer-driven semantic frameworks. While significant progress has been made in automating grading and classification tasks, relatively limited attention has been given to structured, syllabus-grounded question paper validation systems. This chapter examines prior approaches and identifies the research gaps addressed by the proposed work.

The reviewed literature is systematically organized by clustering existing works based on their solution approaches, namely early rule-based and machine learning systems, transformer-based educational NLP models, Bloom's Taxonomy classification techniques, semantic similarity and vector retrieval frameworks, Retrieval-Augmented Generation approaches, pipeline evaluation and benchmarking frameworks, adaptive question generation systems, and agent orchestration and Model Context Protocol patterns. Each cluster is analyzed in depth to identify strengths, limitations, and open research challenges that motivate the proposed work.

2.1 EXISTING WORK

2.1.1 Early Automated Assessment Systems

Initial research in automated academic evaluation was primarily centered around rule-based and keyword-driven approaches. These systems relied on handcrafted rules, predefined templates, and surface-level keyword matching to assess textual content. Although such methods introduced a degree of automation, they lacked contextual understanding and were highly sensitive to linguistic variations. Minor changes in phrasing often led to incorrect classifications, limiting their robustness and generalizability.

As computational techniques evolved, classical machine learning models began replacing rigid rule-based systems. Algorithms such as Support Vector Machines, Naive Bayes classifiers, and Logistic Regression were applied to tasks including short-answer grading and question classification. These models improved adaptability by learning from data rather than depending entirely on manually designed rules. However, they required extensive feature engineering, and their performance was heavily influenced by the quality and domain specificity of extracted features. When applied across different subjects or

academic contexts, their effectiveness declined due to limited semantic generalization.

Despite these advancements, early automated systems primarily focused on evaluating student responses rather than validating the structural and academic quality of examination questions themselves. As a result, aspects such as syllabus alignment, Course Outcome mapping, and cognitive-level distribution remained largely dependent on manual review.

2.1.2 Transformer-Based Models in Educational NLP

The introduction of transformer architectures marked a significant turning point in Natural Language Processing. Bidirectional Encoder Representations from Transformers (BERT) and its variants enabled deep contextual understanding by modeling bidirectional relationships within text. Unlike earlier models that relied on shallow feature extraction, transformer-based systems generate dense contextual embeddings that capture semantic nuance and long-range dependencies.

In educational domains, transformer models have been successfully applied to automated essay grading, short-answer scoring, question classification, cognitive-level detection, and semantic similarity measurement. Their ability to encode contextual meaning allows them to outperform traditional machine learning approaches across multiple evaluation tasks. By leveraging pre-training on large corpora followed by domain-specific fine-tuning, these models achieve high accuracy in classification and similarity-based tasks.

However, most transformer-based assessment systems operate as black-box classifiers. While they provide improved predictive performance, they often lack interpretability and do not explicitly verify whether a question falls within prescribed syllabus boundaries. In high-stakes academic environments, explainability, determinism, and syllabus compliance are critical requirements. The absence of structured validation mechanisms in many transformer-based systems limits their suitability for institutional examination workflows.

2.1.3 Bloom's Taxonomy Classification Using NLP

Bloom's Taxonomy provides a hierarchical framework for categorizing educational objectives into cognitive levels ranging from Remember to Create. Automated classification of questions according to Bloom levels has gained increasing attention within educational NLP research. By analyzing linguistic features, action verbs, and contextual complexity, transformer-based models have been fine-tuned to perform multi-class cognitive classification.

Recent studies demonstrate that contextual embeddings significantly improve Bloom-level detection compared to rule-based verb matching approaches. Instead of relying solely on explicit verbs such as "define" or "analyze," transformer models capture deeper semantic cues that reflect cognitive demand.

In the proposed system, a fine-tuned DeBERTa-v3-base model is employed for Bloom classification. The best fold checkpoint achieved a classification accuracy of 82.44% and a Macro F1 of 0.8499 during validation, demonstrating the suitability of transformer architectures for cognitive-level prediction. Nevertheless, Bloom classification alone does not guarantee syllabus compliance or Course Outcome alignment. In many existing systems, cognitive classification is treated as an isolated task rather than as part of an integrated academic validation framework.

2.1.4 Semantic Similarity and Vector Retrieval

The development of sentence embedding models such as Sentence-BERT has enabled efficient semantic similarity computation by representing textual content as dense vectors in high-dimensional space. These embeddings allow semantic comparison through cosine similarity, capturing conceptual closeness beyond exact keyword matching.

Vector databases such as FAISS have further enhanced the scalability of similarity-based retrieval systems. By enabling fast nearest-neighbor search over large embedding collections, FAISS supports efficient retrieval in document search, topic alignment, and knowledge-grounded applications. Retrieval-based architectures have become foundational in modern NLP systems, particularly in information retrieval and question-answering domains.

However, in the context of academic assessment systems, vector retrieval has rarely been used to enforce strict syllabus boundaries. Most implementations focus on retrieving semantically similar content without incorporating deterministic domain gating or course-level isolation. Consequently, retrieved content may extend beyond prescribed academic scope.

The proposed system addresses this limitation by implementing course-isolated FAISS indices and domain-level validation mechanisms. By restricting retrieval operations to embeddings derived exclusively from the selected course, the system prevents cross-domain semantic leakage and ensures adherence to academic boundaries.

2.1.5 Retrieval-Augmented Generation (RAG)

Large Language Models have demonstrated strong capabilities in generating coherent and contextually rich text. However, when used without grounding, they are prone to hallucination and may generate content beyond prescribed syllabus scope. This limitation poses a significant challenge in academic settings where accuracy and compliance are essential.

Retrieval-Augmented Generation mitigates this issue by combining retrieval mechanisms with generative models. In RAG architectures, relevant contextual information is first retrieved from a curated knowledge base. The generative model then conditions its

output on this retrieved context, producing responses that are grounded in verified source material. This approach has been successfully applied in open-domain question answering, knowledge-grounded conversational systems, and educational content generation.

Despite its promise, limited research has explored the application of RAG for controlled examination question generation under strict syllabus constraints. Many existing implementations operate in open-domain environments where retrieval scope is broad and loosely defined. Structured academic validation requirements, such as Course Outcome alignment and Bloom-level constraints, are rarely integrated into the generation process.

The proposed system applies RAG within a tightly controlled academic framework. Retrieval is restricted to syllabus and textbook embeddings belonging to the selected course. Generation is constrained by specified Course Outcome and Bloom-level requirements, and generated questions are re-evaluated through the same validation pipeline used for manual questions. This closed-loop design enhances reliability and ensures academic compliance.

2.1.6 Pipeline Evaluation and Benchmarking Frameworks

Evaluating the end-to-end accuracy of NLP pipelines in educational settings presents unique challenges. Unlike isolated classification tasks, multi-stage pipelines introduce compounding errors across retrieval, matching, and classification stages, making holistic benchmarking essential for reliable deployment. Prior works in information retrieval and question-answering evaluation have proposed metrics such as Mean Reciprocal Rank, NDCG, and F1-based answer matching to assess pipeline-level performance. However, these metrics are designed for general-domain retrieval tasks and do not account for the specific constraints of academic validation, such as Bloom-level adjacency tolerance or semantic topic matching thresholds.

In academic NLP systems, evaluation has typically been performed at the component level, measuring classification accuracy of individual models in isolation rather than assessing the integrated pipeline behavior. As a result, component-level accuracy scores often do not reflect real-world performance when models are composed into multi-stage workflows. The absence of standardized benchmarking protocols for academic validation pipelines makes it difficult to compare systems across studies or identify systematic failure patterns at specific cognitive levels or Course Outcome boundaries.

The proposed system addresses this gap by implementing a dedicated pipeline validation module that evaluates end-to-end prediction accuracy against labelled ground-truth datasets using three complementary matching strategies: relaxed Bloom matching to account for cognitive boundary ambiguity, semantic cosine topic matching to handle phrasing variation, and exact CO matching to enforce institutional traceability requirements. Both a user-facing metrics dashboard and a developer-level CLI diagnostic tool are pro-

vided, enabling systematic identification and correction of pipeline-level errors across all stages.

2.1.7 Adaptive Question Generation and Gap Analysis

Automated question generation has been explored extensively in educational NLP, with approaches ranging from template-based generation to transformer-based generative models. Early systems employed predefined templates and slot-filling mechanisms to produce questions from structured knowledge sources, but these approaches lacked flexibility and were limited to narrow domain coverage. More recent transformer-based systems such as T5 and GPT variants have demonstrated the ability to generate fluent and contextually appropriate questions from free-form text, significantly improving generation quality and diversity.

However, existing question generation systems predominantly focus on producing questions from given passages without considering whether the generated content aligns with specific curricular objectives, Course Outcomes, or cognitive-level requirements. The integration of gap analysis — systematically detecting deficiencies in Bloom-level distribution, Course Outcome coverage, and syllabus unit representation across an evaluated question paper — remains largely unexplored in prior literature. Most existing tools provide only post-hoc analysis reports without offering actionable generation-based remediation.

Furthermore, prior adaptive assessment systems have focused primarily on student-facing personalization, adjusting question difficulty based on learner performance, rather than on faculty-facing paper quality improvement. The challenge of mapping detected coverage gaps to appropriate generation targets without relying on hardcoded positional assumptions, and of enforcing domain-specific concept usage in generated questions to prevent generic output, has not been systematically addressed in existing work.

The proposed system addresses these limitations by implementing a multi-dimensional gap detection layer that independently identifies Bloom-level imbalances, missing or underrepresented Course Outcomes, and uncovered syllabus units. For each detected gap, the system applies semantically grounded generation using FAISS-retrieved and cross-encoder-reranked syllabus context supplied to the Groq LLM, with explicit prompt constraints enforcing domain-specific concept usage. This closed-loop design, where generated questions are re-evaluated through the same validation pipeline before presentation to faculty, represents a significant advancement over existing open-loop generation approaches.

2.1.8 Agent Orchestration and Model Context Protocol

The concept of agent orchestration in NLP systems refers to the design pattern in which a central controller, termed an agent, manages the invocation of modular, context-aware tools to accomplish complex multi-step tasks. The Model Context Protocol (MCP) formalizes this pattern by defining how an agent receives user intent, selects appropriate tools, passes structured context to each tool, and aggregates the results into a coherent output. This pattern has gained prominence in recent AI system design, particularly in applications requiring deterministic coordination of heterogeneous components such as retrieval engines, classification models, and generative modules.

In educational NLP systems, prior works have rarely adopted explicit agent orchestration architectures. Most existing systems implement monolithic pipelines where components are tightly coupled and not independently callable. This limits modularity, testability, and the ability to extend or replace individual components without affecting the entire system. The absence of a structured tool-use pattern makes it difficult to maintain deterministic execution order, enforce domain-level validation gates, or support parallel execution of independent modules such as retrieval and classification.

The proposed system addresses this gap by implementing the MCP tool-use pattern natively. A central AgentOrchestrator class receives user requests and coordinates the sequential or parallel invocation of six modular tools — Retrieval, Classification, Matching, Generation, Evaluation, and Export — each receiving structured context comprising the user identifier, domain name, and question content. This design ensures that tool execution is fully traceable, independently testable, and extensible without modifying the orchestration layer, representing a significant architectural advancement over the tightly coupled pipelines found in existing academic assessment systems.

2.2 OBSERVATIONS FROM THE EXISTING WORK

The reviewed literature demonstrates that transformer-based representations significantly enhance academic assessment by capturing contextual and semantic dependencies in educational content. Unsupervised and retrieval-based learning approaches are particularly promising for syllabus-grounded validation, as they eliminate dependence on manually curated rule sets. Autoencoder-based and embedding-based models effectively learn semantic patterns and detect deviations through similarity-based comparison.

However, most existing approaches either rely on isolated classification models or treat retrieval as a secondary feature source. Many frameworks employ keyword-centric or task-specific models, failing to explicitly represent the full academic constraint space that includes syllabus compliance, Course Outcome mapping, cognitive-level distribution, and difficulty estimation simultaneously. Pipeline-level benchmarking, adaptive gap-driven

generation, and structured MCP-style agent orchestration remain largely unaddressed in prior work, leaving significant room for improvement in end-to-end academic validation systems.

2.3 LIMITATIONS OF EXISTING WORK

Despite substantial progress, current automated assessment systems exhibit notable limitations. Supervised dependency restricts generalization across diverse academic domains, while isolated classification models introduce inconsistency across evaluation workflows. Most existing frameworks lack mechanisms for strict syllabus-bound validation and do not incorporate deterministic domain gating. Bloom classification and syllabus mapping are often implemented independently without integrated cross-verification. Additionally, transformer-based systems frequently lack explainability, making it difficult to justify evaluation decisions in institutional contexts. Furthermore, generation and validation are typically treated as separate tasks, with minimal integration between them. End-to-end pipeline benchmarking against labelled ground-truth datasets is rarely implemented, and adaptive gap analysis with automated remediation generation remains an open research challenge in academic assessment systems. Additionally, existing systems do not adopt structured agent orchestration patterns such as the Model Context Protocol, resulting in tightly coupled, non-modular pipelines that are difficult to extend, test independently, or maintain in institutional deployment contexts.

2.4 CHALLENGES

Key challenges identified from the literature include the absence of standardized labeled datasets for Bloom-level classification across diverse academic domains, difficulty in enforcing strict syllabus boundaries using general-purpose language models, and ensuring consistent evaluation behavior between single-question and batch-PDF workflows. Capturing the full academic constraint space — comprising syllabus compliance, Course Outcome alignment, cognitive classification, and difficulty estimation — within a unified deterministic framework remains a critical research challenge. Additionally, integrating controlled generation within a closed validation loop while maintaining reproducibility and academic integrity introduces significant architectural complexity. Benchmarking multi-stage pipeline accuracy using metrics that appropriately handle semantic variation in topic matching and cognitive-level adjacency in Bloom classification presents further methodological challenges not addressed by existing evaluation frameworks. Designing a modular, MCP-compliant agent orchestration layer that enforces deterministic execution while remaining independently testable and extensible introduces additional systems-level complexity not addressed in prior academic NLP research.

2.5 SUMMARY OF LITERATURE SURVEY

This chapter reviewed existing automated academic assessment approaches with a focus on transformer-based NLP models, Bloom classification techniques, vector retrieval systems, Retrieval-Augmented Generation frameworks, pipeline evaluation methodologies, adaptive question generation systems, and agent orchestration patterns. While prior works demonstrate the effectiveness of contextual embeddings and retrieval-based architectures, they exhibit limitations in supervised dependency, integrated syllabus-bound validation, pipeline-level benchmarking, closed-loop generation with gap-driven remediation, and structured MCP-style tool orchestration. These research gaps motivate the proposed work, which introduces an Agent-Orchestrated Automated Question Paper Validation and Generation System integrating semantic retrieval, hybrid cross-encoder validation, fine-tuned DeBERTa-v3-base Bloom classification, dedicated pipeline benchmarking, adaptive gap analysis, and syllabus-constrained RAG generation within a unified deterministic pipeline following the Model Context Protocol tool-use pattern.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system, termed the **Agent-Orchestrated Automated Question Paper Validation and Generation System**, is a hybrid NLP framework designed to perform end-to-end syllabus compliance validation, Bloom's Taxonomy cognitive classification, Course Outcome mapping, pipeline-level accuracy benchmarking, and adaptive question generation from examination question papers. Unlike traditional assessment tools that rely on rule-based heuristics or isolated classifiers, the proposed architecture models academic knowledge as structured semantic embeddings and validates questions through a deterministic multi-stage pipeline grounded exclusively in syllabus content. The architecture is composed of seven primary modules: a Course Knowledge Setup module, a Question Input and Preprocessing module, a RAG Retrieval and Topic Matching module, a Bloom Taxonomy Classification module, a Scoring and Output Generation module, an Evaluation Metrics and Pipeline Validation module, and an Adaptive Question Generation and Gap Analysis module. These modules are implemented as modular, context-aware tools invoked by a central AgentOrchestrator following the Model Context Protocol tool-use pattern, where each tool receives structured context comprising the user identifier, domain name, and question content, and returns structured results that are aggregated by the orchestrator into the final evaluation output.

At a high level, official syllabus documents and reference textbooks are first parsed, chunked, and embedded using the Sentence-BERT model `all-MiniLM-L6-v2` into course-isolated FAISS vector indices stored in MongoDB GridFS. When a faculty member submits a question or uploads a full examination paper, the system initiates a deterministic pipeline that performs syllabus-grounded topic matching through bi-encoder filtering and cross-encoder reranking, classifies the cognitive level using a fine-tuned DeBERTa-v3-base model, and computes difficulty and strength scores. A pipeline validation module benchmarks end-to-end prediction accuracy against labelled ground-truth datasets using relaxed Bloom matching, semantic cosine topic matching, and exact CO matching strategies. An adaptive gap analysis module subsequently analyses evaluated question papers to detect Bloom-level imbalances, missing or underrepresented Course Outcomes, and uncovered syllabus units, triggering targeted question generation via the Groq LLM or guided faculty correction workflows. All modules operate under a centralized FastAPI-based Agent Orchestrator that enforces strict domain isolation and deterministic execution order across all workflows.

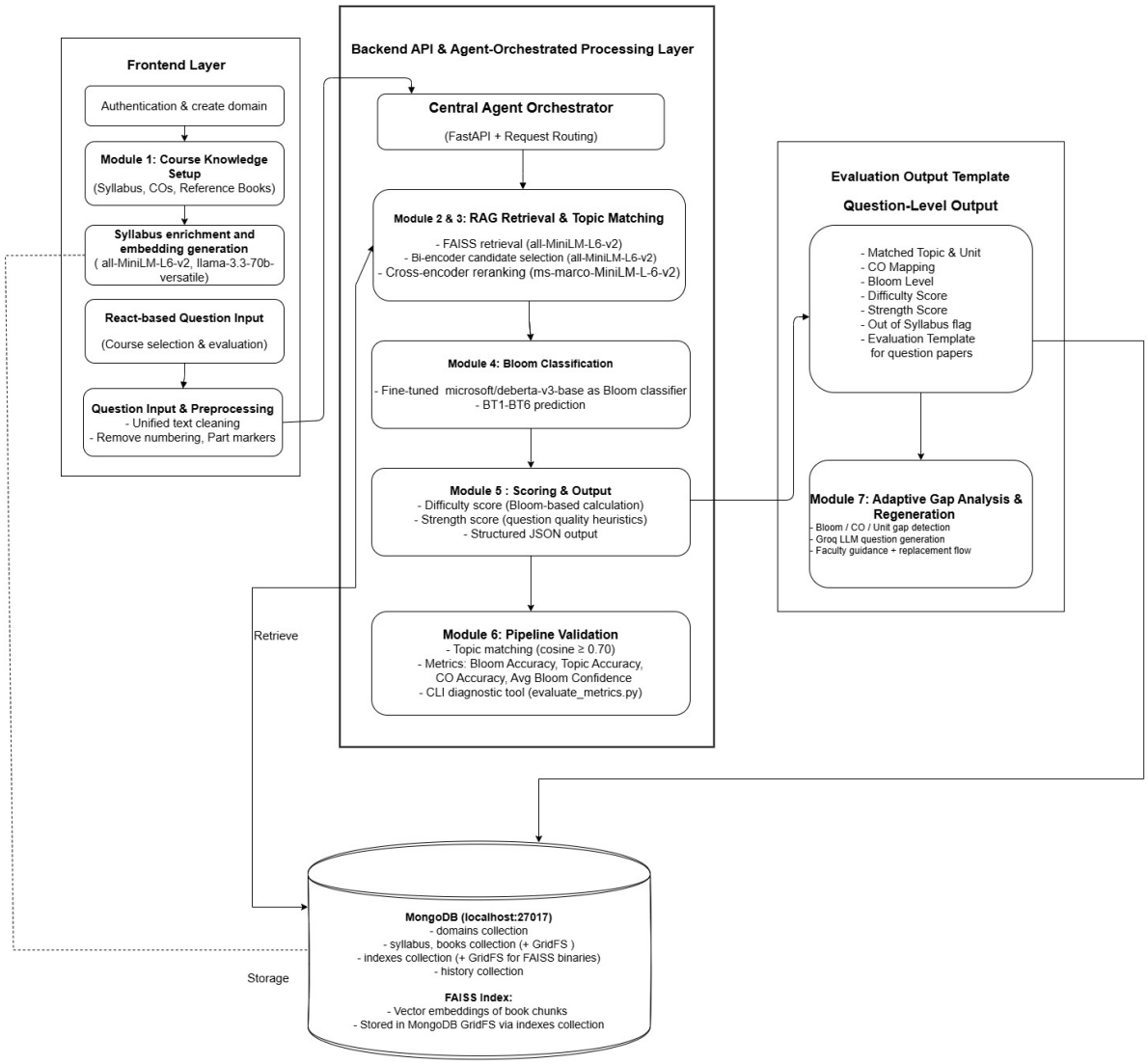


Figure 3.1: Overall System Architecture of the Proposed Agent-Orchestrated Question Paper Validation and Generation System

3.2 COURSE KNOWLEDGE SETUP MODULE

The Course Knowledge Setup module forms the foundational layer of the proposed system. It is responsible for transforming raw academic documents into a structured, machine-interpretable semantic repository that serves as the grounding source for all downstream validation and generation operations.

The module accepts two primary input categories: official syllabus documents containing instructional units (Unit I–V), subtopics, and Course Outcomes (CO1–CO5), and reference textbook PDFs containing domain-specific academic content. The syllabus document is parsed and normalized into clearly defined instructional units and atomic subtopics. Each unit is mapped explicitly to its corresponding Course Outcomes, forming a hierarchical academic structure that enables precise topic-to-CO alignment during

evaluation.

Reference textbooks are processed by extracting textual content and dividing it into semantically coherent chunks of bounded token length. This chunking strategy preserves contextual continuity while maintaining computational efficiency during embedding and retrieval operations. An additional syllabus enrichment step employs `llama-3.3-70b-versatile` to match book chunks against syllabus topics and generate enriched subtopic descriptions, significantly enhancing the semantic depth of the knowledge base beyond what the raw syllabus document provides.

All textual elements, including syllabus units, enriched subtopics, and textbook chunks, are embedded using the Sentence-BERT model `all-MiniLM-L6-v2`, generating 384-dimensional dense vector representations that capture contextual semantic meaning beyond surface-level keyword similarity. The generated embeddings are indexed using FAISS `IndexFlatIP` with cosine similarity, supporting over 6500 vectors per domain. The serialized FAISS index and associated metadata are stored in MongoDB GridFS, ensuring persistent, domain-isolated storage. Each academic course maintains independent syllabus JSON structures, enriched topic representations, FAISS vector indices, and evaluation history collections, preventing any cross-domain semantic interference.

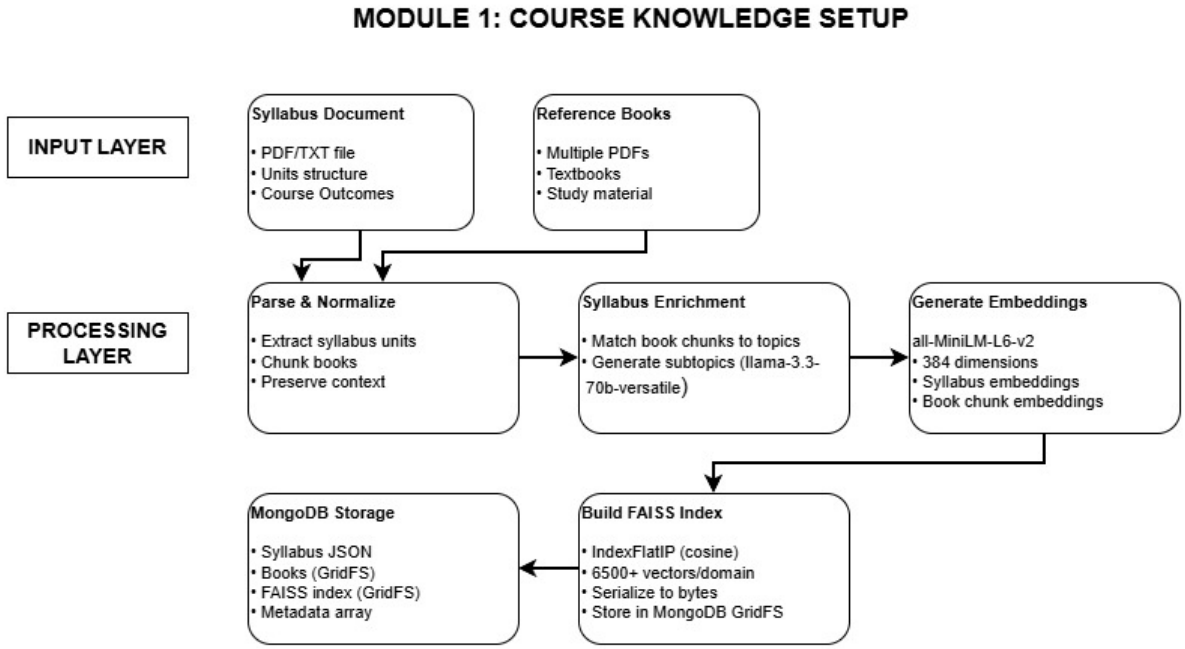


Figure 3.2: Module 1: Course Knowledge Setup

By exporting all intermediate products to a structured MongoDB storage layer – serialized FAISS indices via GridFS, syllabus JSON, book PDFs, and metadata arrays – the module decouples document-specific parsing from subsequent retrieval, validation, and generation experiments, enabling reproducible evaluation across multiple academic domains.

3.3 QUESTION INPUT AND PREPROCESSING MODULE

The Question Input and Preprocessing module handles all incoming evaluation requests and standardizes them into a consistent, evaluation-ready format before passing them to downstream pipeline stages. The module supports two input modes: single-question evaluation, where a faculty member submits an individual question as raw text, and batch PDF evaluation, where an entire examination question paper is uploaded for bulk processing.

For PDF inputs, the system employs a structured parser (`PDFQuestionParser`) capable of detecting sectional boundaries such as Part A, Part B, and Part C. The parser identifies question numbering patterns, handles OR-type alternatives such as questions 19a and 19b, splits compound questions by question number, and removes trailing marks annotations. Each extracted question is treated as an independent evaluation unit.

A unified text cleaning function is then applied to all inputs, regardless of whether they originate from single-question or batch mode. This function removes leading numbering patterns (e.g. Q.1, 3., 16a), part markers, bullet symbols, and marks-related annotation artifacts. Unicode normalization (NFKC) is applied to ensure consistent encoding across diverse input sources, while whitespace collapsing removes redundant formatting inconsistencies. The clean question output from this module is forwarded simultaneously to the RAG Retrieval and Topic Matching module and the Bloom Taxonomy Classification module.

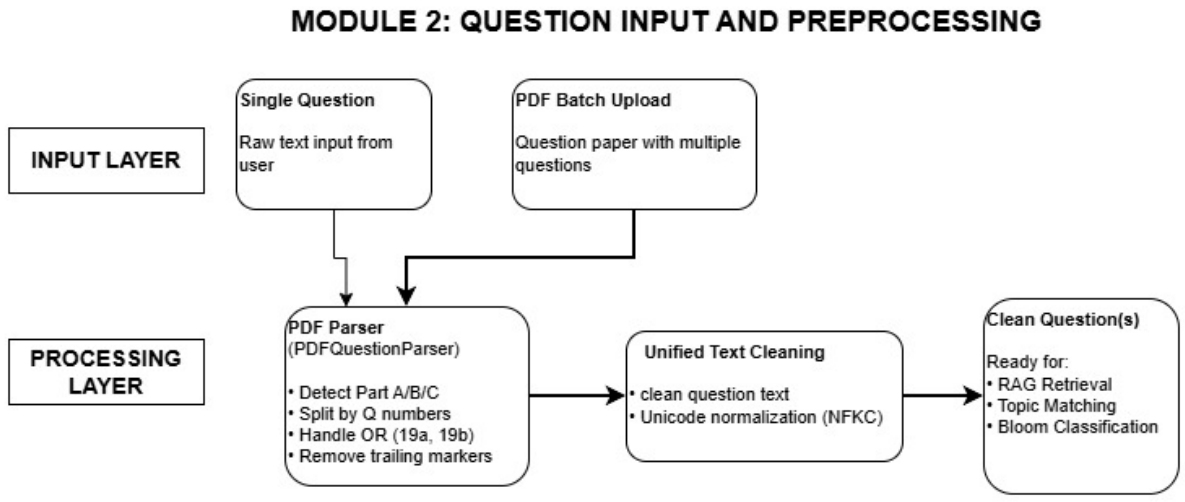


Figure 3.3: Module 2: Question Input and Preprocessing

This unified preprocessing design guarantees identical cleaning behavior across all evaluation workflows, eliminating the inconsistencies previously observed between single-question and batch processing modes and ensuring that evaluation results are fully reproducible regardless of the input pathway used.

3.4 RAG RETRIEVAL AND TOPIC MATCHING MODULE

The RAG Retrieval and Topic Matching module constitutes the semantic backbone of the validation pipeline. It implements a two-stage hybrid retrieval mechanism that combines bi-encoder filtering with cross-encoder reranking to achieve both computational efficiency and high contextual accuracy in syllabus alignment.

In Stage 1, the cleaned question text is embedded using `all-MiniLM-L6-v2` to produce a 384-dimensional query vector. This embedding is searched against the domain-specific FAISS index loaded from MongoDB GridFS. The top 10 most semantically similar textbook chunks are retrieved along with their source metadata, providing contextual grounding evidence for topic validation and reducing the risk of alignment decisions based purely on classifier outputs.

In Stage 2, the enriched syllabus topics are evaluated against the question. Each topic's `topic_text` field is formed by concatenating the topic name with its enriched subtopics generated during knowledge base construction. The cross-encoder model `ms-marco-MiniLM-L-6-v2` jointly encodes the question and each candidate topic text, computing fine-grained contextual relevance scores. Raw logits are converted to probabilities using sigmoid transformation, and relative rank-based gating logic is applied to determine whether the best candidate satisfies the acceptance threshold. If validated, the matched topic is mapped to its corresponding instructional unit (Unit I–V), unit title, and Course Outcomes (CO1–CO5), along with a confidence score and an out-of-syllabus flag. If no candidate meets the acceptance criteria, the question is flagged as out-of-syllabus and excluded from Bloom classification and scoring.

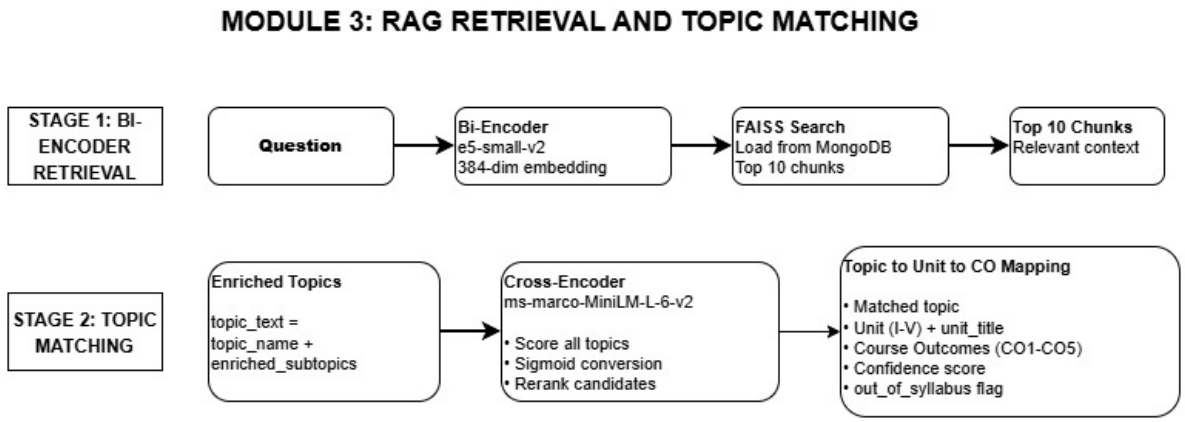


Figure 3.4: Module 3: RAG Retrieval and Topic Matching

This hybrid approach significantly reduces false out-of-syllabus classifications while maintaining robustness against superficial lexical overlap between question text and unrelated syllabus topics, a key limitation of single-stage bi-encoder retrieval systems.

3.5 BLOOM TAXONOMY CLASSIFICATION MODULE

The Bloom Taxonomy Classification module categorizes each validated question into one of six cognitive levels defined by Bloom's Taxonomy, ranging from BT1 (Remember) to BT6 (Create). A fine-tuned DeBERTa-v3-base model (`final_bloom_model`) is employed for multi-class classification, selected on the basis of Fold-2 cross-validation performance.

The input question is first tokenized using the DeBERTa tokenizer, producing token ID sequences suitable for transformer inference. The fine-tuned model processes these sequences and outputs raw logits across all six Bloom levels. A softmax function converts the logits into a probability distribution, and the class with the highest probability is selected as the predicted Bloom level. The associated probability value serves as the Bloom confidence score, which is propagated downstream to the scoring module and the pipeline validation module.

The model operates in deterministic inference mode using `model.eval()` and `torch.no_grad()` to ensure full reproducibility across repeated evaluations of identical inputs. The selected Fold-2 best model achieved a validation accuracy of 82.44% and a Macro F1 of 0.8499, with a five-fold average Macro F1 of 0.8413, confirming the suitability of the fine-tuned DeBERTa-v3-base architecture for cognitive-level prediction in academic assessment contexts. This module operates independently within the orchestration pipeline and may execute concurrently with topic matching, improving overall system throughput for batch evaluation workflows.

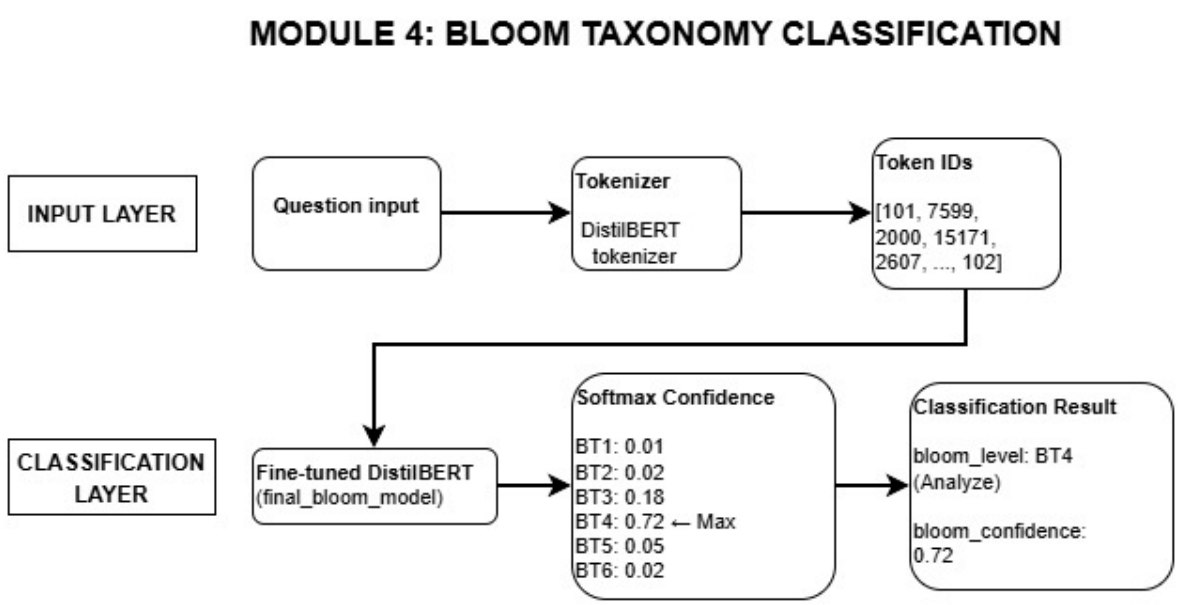


Figure 3.5: Module 4: Bloom Taxonomy Classification

3.6 SCORING AND OUTPUT GENERATION MODULE

The Scoring and Output Generation module aggregates the results produced by the RAG Retrieval, Topic Matching, and Bloom Classification modules to compute structured evaluation outputs for each question. The aggregation layer combines the topic result – comprising unit, unit title, matched topic, and Course Outcomes – with the Bloom result comprising BT level and confidence score.

A difficulty score is computed deterministically using a fixed Bloom-level weight mapping, where BT1 corresponds to a difficulty of 0.15 and BT6 corresponds to 1.00, with intermediate levels mapped proportionally to reflect increasing cognitive demand. A strength score is derived from heuristic quality indicators including question length, presence of analytical action verbs, and semantic content density, providing a quality-oriented metric independent of cognitive level.

The complete evaluation record is compiled into a structured JSON object containing all relevant fields: matched unit, unit title, topic, Course Outcomes, Bloom level, Bloom confidence, difficulty score, strength score, and the top retrieved book chunks as supporting context. This structured output is persisted to MongoDB's history collection, enabling evaluation history tracking, per-session review, batch report export, and undo operations. Export options include PDF and DOCX formats containing the complete CO-BT mapping report for institutional submission and review.

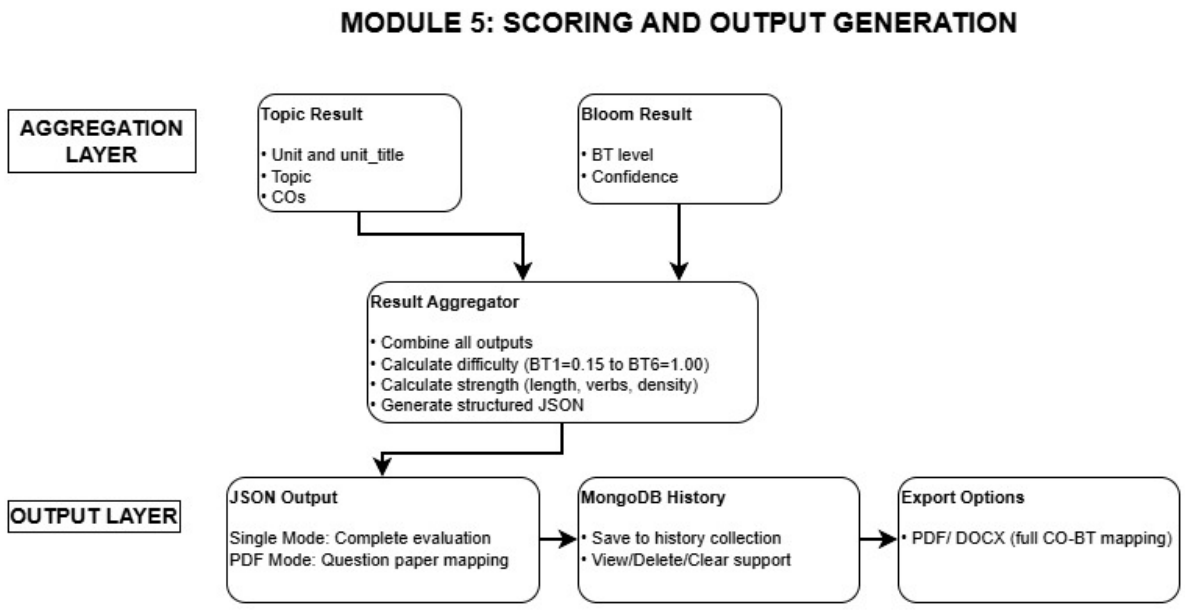


Figure 3.6: Module 5: Scoring and Output Generation

In single-question mode, the complete evaluation for one question is returned immediately. In batch PDF mode, a full question paper mapping is generated with per-question evaluations aggregated into a structured report, ensuring consistent and reproducible

outputs across both operational modes.

3.7 EVALUATION METRICS AND PIPELINE VALIDATION MODULE

The Evaluation Metrics and Pipeline Validation module measures the end-to-end performance of the validation pipeline using a labelled dataset of questions with ground-truth annotations for topic, Bloom level, and Course Outcome. This module provides both a user-facing metrics dashboard and a developer-level CLI diagnostic tool, enabling performance monitoring at both operational and research levels.

The input to this module is a JSON dataset where each entry contains the question text, true topic, true Bloom level, and true Course Outcome. The full evaluation pipeline (`evaluate_question()`) is executed independently for each entry, returning the predicted Bloom level, predicted topic, predicted CO, and Bloom confidence score for comparison against ground truth.

Three matching strategies are applied to assess prediction quality. Bloom matching employs a ± 1 relaxed criterion, where a prediction is considered correct if it falls within one Bloom level of the ground truth; both strict (exact) and relaxed accuracy are computed, with the relaxed metric serving as the primary indicator given the overlapping cognitive characteristics of adjacent levels. Topic matching is performed semantically by encoding both the predicted and true topic using the existing `all-MiniLM-L6-v2` bi-encoder and computing cosine similarity; a score of ≥ 0.70 is treated as correct, and the raw similarity score is stored per question for transparency. CO matching requires a strict exact match between the top-ranked predicted CO and the ground truth, reflecting the institutional requirement for precise outcome traceability.

The metrics computed by this module include strict and relaxed Bloom accuracy, topic accuracy at the 0.70 cosine threshold, CO accuracy, average, minimum, and maximum topic similarity scores, average Bloom confidence, and a per-question breakdown showing all predicted versus true values with correctness indicators. The CLI tool (`evaluate_metrics.py`) provides complete diagnostic output including Bloom confusion patterns (e.g. BT2→BT3: 3 occurrences) and CO mismatch analysis for developer-level inspection and iterative model improvement.

3.8 ADAPTIVE QUESTION GENERATION AND GAP ANALYSIS MODULE

The Adaptive Question Generation and Gap Analysis module analyses an evaluated question paper, detects coverage gaps across Bloom levels, Course Outcomes, and syllabus

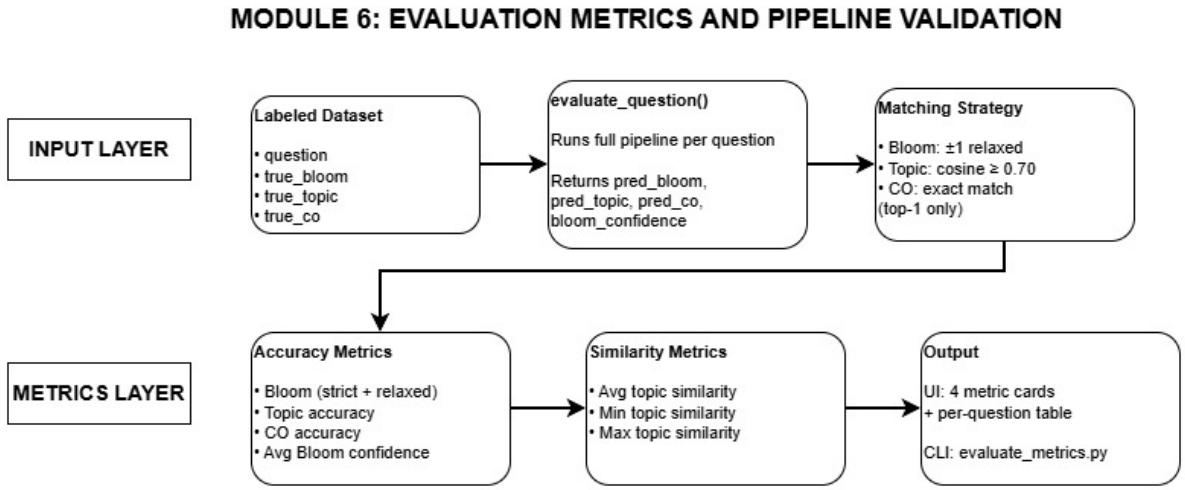


Figure 3.7: Module 6: Evaluation Metrics and Pipeline Validation

units, and generates or guides targeted replacement questions to improve the overall cognitive balance and topical completeness of the examination.

The gap detection layer operates across three dimensions. Bloom gap detection is triggered when BT1 and BT2 questions collectively exceed 60 percent of the paper, or when higher-order questions (BT4 and above) fall below 20 percent of the total. The weakest unit – defined as the unit with the lowest average Bloom level across its mapped questions – is identified as the primary replacement target. CO gap detection counts the top-ranked predicted CO per question; a CO is flagged as missing if its count is zero, or underrepresented if its count is below two. CO-to-unit mapping is performed semantically by encoding each CO description and comparing it against unit texts using cosine similarity, correctly resolving ambiguous mappings without relying on hardcoded positional assumptions. Unit gap detection flags any syllabus unit with no questions mapped to it, indicating a structural gap in the paper's topical coverage.

The generation layer employs two distinct strategies depending on the gap type. For Bloom gaps, the system provides guidance only: it presents the target Bloom level, a curated list of recommended action verbs, and a domain-specific example question structure, with the professor retaining authorial control through a "Generate Question" or "Write My Own" interface. For CO and unit gaps, auto-generation is triggered by dispatching a structured prompt to the Groq LLM (`llama-3.3-70b-versatile`). The prompt includes the subject name, unit title, syllabus topics, enriched subtopics retrieved via FAISS and cross-encoder reranking, CO description, and target Bloom action verbs. Generic phrasing is explicitly forbidden in the prompt, requiring domain-specific concept usage throughout all generated questions.

The replacement flow allows the professor to select which existing low-level (BT1/BT2) question to replace from a picker list, review or edit the generated ques-

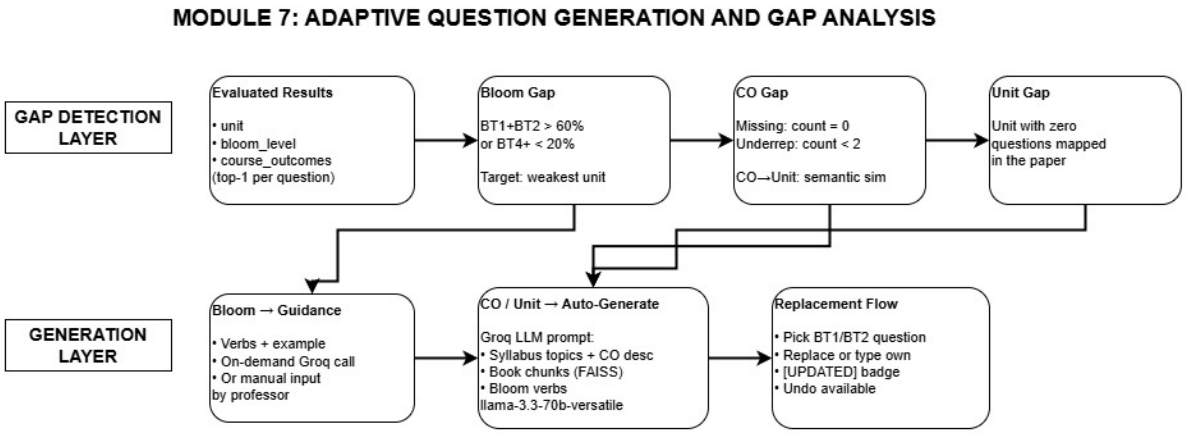


Figure 3.8: Module 7: Adaptive Question Generation and Gap Analysis

tion inline, and confirm the replacement via API. Confirmed replacements are reflected immediately in the Evaluation tab with an `[UPDATED]` badge, and the original question is preserved for comparison with a full undo option available.

3.9 SUMMARY

The proposed architecture transitions academic question paper validation from rule-based classification toward a knowledge-grounded, semantically-driven evaluation framework built on the Model Context Protocol tool-use pattern, where a central AgentOrchestrator coordinates six modular tools — Retrieval, Classification, Matching, Generation, Evaluation, and Export — in a deterministic, traceable execution sequence. By modeling academic content as structured embeddings and routing evaluation through a deterministic multi-stage pipeline, the system captures syllabus compliance, cognitive distribution, and outcome coverage dimensions that are not visible in traditional isolated classifier approaches. The unified preprocessing design eliminates behavioral inconsistencies across evaluation modes, while course-isolated FAISS indices prevent cross-domain semantic interference. The integration of adaptive gap analysis and controlled RAG-based generation within the same validation loop further demonstrates the system's applicability to practical institutional examination workflows, providing faculty with a transparent, explainable, and reproducible tool for OBE-compliant question paper design.

CHAPTER 4

IMPLEMENTATION

This chapter details the practical implementation of the proposed **Agent-Orchestrated Automated Question Paper Validation and Generation System**, describing the software environment, knowledge base construction pipeline, semantic retrieval and topic matching procedures, Bloom classification model training, scoring and output generation, pipeline validation benchmarking, and the adaptive gap analysis and question generation module. The implementation follows a structured seven-stage pipeline: academic knowledge base setup, question preprocessing, RAG retrieval and topic matching, Bloom classification, scoring and output aggregation, evaluation metrics computation, and adaptive question generation with gap analysis.

4.1 EXPERIMENTAL ENVIRONMENT

All modules were implemented in Python using the `FastAPI` framework as the central agent orchestrator. Semantic embedding and retrieval were implemented using `sentence-transformers` with the `all-MiniLM-L6-v2` model, and vector similarity search was performed using the `faiss-cpu` library with `IndexFlatIP`. Cross-encoder reranking was implemented using the `cross-encoder/ms-marco-MiniLM-L-6-v2` model from `sentence-transformers`. Bloom classification was implemented using the `transformers` library with a fine-tuned DeBERTa-v3-base model. Syllabus enrichment and adaptive question generation employed the Groq API with the `llama-3.3-70b-versatile` model. Persistent storage was handled using `pymongo` with MongoDB running locally on port 27017, with large binary objects including serialized FAISS indices and reference book PDFs managed through MongoDB GridFS. The React-based frontend communicated with the FastAPI backend through REST endpoints. The system implements the Model Context Protocol tool-use pattern natively, with the FastAPI-based AgentOrchestrator acting as the central agent that receives user intent and coordinates the invocation of modular tools — FAISS retrieval, DeBERTa-v3-base classification, cross-encoder topic matching, Groq LLM generation, metrics evaluation, and PDF/DOCX export — each independently callable as internal services under the centralized orchestration layer.

To ensure reproducibility of Bloom classification inference, `model.eval()` and `torch.no_grad()` contexts were enforced on all transformer inference calls, and deterministic behavior was maintained through consistent tokenizer and model loading procedures. All modules were designed to be independently callable as internal services under the centralized orchestration layer, enabling modular testing and integration.

4.2 KNOWLEDGE BASE CONSTRUCTION PIPELINE

The knowledge base construction stage transforms raw academic documents into a structured semantic repository. This stage populates the MongoDB collections and GridFS stores that are subsequently used by all retrieval, validation, and generation modules.

4.2.1 Syllabus Parsing and Normalization

The official syllabus document is uploaded as a PDF or plain-text file and parsed into a structured JSON representation. Each instructional unit (Unit I–V) is extracted along with its title, list of subtopics, and associated Course Outcomes (CO1–CO5). The parsed structure forms a hierarchical academic representation in which each unit node contains its CO mappings and atomic subtopic list. This JSON structure is stored in the `domains` and `syllabus` MongoDB collections, indexed by a unique domain identifier assigned to each academic course.

4.2.2 Reference Book Chunking and Enrichment

Reference textbook PDFs are processed by extracting full text page-by-page and dividing it into semantically coherent chunks of bounded token length. Let L denote the target chunk length in tokens and δ the overlap between consecutive chunks. For a page sequence $\{P_i\}$, the chunk set $\{C_j\}$ is constructed such that each chunk preserves local contextual continuity while remaining within the embedding model's maximum input length.

An additional enrichment step is applied using `llama-3.3-70b-versatile` via the Groq API. For each syllabus topic, the top-k retrieved book chunks are supplied to the LLM alongside the topic name, and the model generates enriched subtopic descriptions that capture domain-specific elaborations not explicitly stated in the syllabus. These enriched subtopics are stored alongside the original syllabus topics and are subsequently used to form the `topic_text` field during cross-encoder reranking.

4.2.3 Embedding Generation and FAISS Index Construction

All textual elements – syllabus unit texts, enriched topic descriptions, and book chunks – are embedded using `all-MiniLM-L6-v2`, which produces 384-dimensional dense vectors. Let $\mathbf{e}_i \in \mathbb{R}^{384}$ denote the embedding of the i-th textual element. The complete embedding matrix $E = [\mathbf{e}_1, \mathbf{e}_2, \ldots, \mathbf{e}_N]^\top$ is indexed using FAISS `IndexFlatIP`, which supports inner-product (cosine) similarity search over more than 6500 vectors per domain:

$$\text{score}(q, i) = \frac{\mathbf{q} \cdot \mathbf{e}_i}{\|\mathbf{q}\| \, \|\mathbf{e}_i\|},$$

where $\mathbf{q}$ is the query embedding. The serialized FAISS index is stored in MongoDB GridFS under the `indexes` collection, keyed by domain identifier, ensuring strict course-level isolation. Each domain maintains its own independent FAISS index, preventing any cross-course semantic interference during retrieval.

Algorithm 1 Knowledge Base Construction

Require: Syllabus document, Course Outcomes, Reference book PDFs

Ensure: Structured domain stored in MongoDB with serialized FAISS index in GridFS

function BUILD_KNOWLEDGE_BASE(syllabus, COs, books)
    units ← ParseAndNormalise(syllabus)
    book_chunks ← ExtractAndChunk(books, L, δ)
    enriched ← GroqLLM_Enrich(book_chunks, units)
    for all topic $t \in$ units **do**
        $\mathbf{e}_t \leftarrow$ SBERT_Embed(t.topic_text $\cup$ enriched[t])
    end for
    for all chunk $c \in$ book_chunks **do**
        $\mathbf{e}_c \leftarrow$ SBERT_Embed(c)
    end for
    $E \leftarrow [\mathbf{e}_1, \ldots, \mathbf{e}_N]^\top$
    index ← FAISS_IndexFlatIP(E)
    GridFS_Store(index, domain_id)
    MongoDB_Store(units, COs, domain_id)
    return domain_id
end function

4.3 QUESTION INPUT AND PREPROCESSING PIPELINE

The preprocessing module standardizes all incoming evaluation requests into a consistent, cleaning-normalized format. Consistent preprocessing is critical because any difference in cleaning between single-question and batch modes would introduce evaluation inconsistencies.

4.3.1 PDF Question Extraction

For batch PDF inputs, the `PDFQuestionParser` extracts raw text using page-level parsing and applies structured sectional detection logic. The parser identifies Part A, Part B, and Part C boundaries, detects question numbering patterns using regular expressions, and splits OR-type alternatives (e.g., questions 19a and 19b) into independent evaluation units. Trailing annotation patterns such as marks labels and sub-question codes are stripped from each extracted question.

4.3.2 Unified Text Cleaning

A unified text cleaning function is applied identically to all inputs regardless of their origin. For a raw input string s, the cleaning pipeline applies the following transformations in sequence:

1. Remove leading numbering patterns: $s \leftarrow$ `re.sub(r'^[\d]+[.)]] ?', '', `s)
2. Remove part markers (e.g., Part A, Part B, Q., Q.No): $s \leftarrow$ `re.sub(part_pattern, '', `s)
3. Apply Unicode normalization: $s \leftarrow$ `unicodedata.normalize('NFKC', `s)
4. Collapse redundant whitespace: $s \leftarrow$ `re.sub(r'\s+', ' ', `s`).strip()`

This sequence guarantees that questions extracted from PDF batch mode and questions entered manually in single-question mode undergo identical normalization, ensuring that evaluation results are fully reproducible regardless of input pathway.

Algorithm 2 Unified Question Preprocessing

Require: Raw question text or PDF file
Ensure: Cleaned and structured question list

```
function PREPROCESS_INPUT(input)
    if input is PDF then
        questions ← PDFQuestionParser(input)
    else
        questions ← {input}
    end if
    for all q ∈ questions do
        q ← RemoveLeadingNumbering(q)
        q ← RemovePartMarkers(q)
        q ← NormaliseUnicode(q)                ▷ NFKC
        q ← CollapseWhitespace(q)
    end for
    return questions
end function
```

4.4 RAG RETRIEVAL AND TOPIC MATCHING IMPLEMENTATION

The RAG Retrieval and Topic Matching module constitutes the semantic core of the validation pipeline and is implemented as a two-stage hybrid mechanism combining fast bi-encoder retrieval with precise cross-encoder reranking.

4.4.1 Stage 1: Bi-Encoder FAISS Retrieval

The cleaned question q is embedded using `all-MiniLM-L6-v2` to produce a 384-dimensional query vector $\mathbf{q}$. An inner-product search is performed against the domain-specific FAISS `IndexFlatIP` structure loaded from MongoDB GridFS:

$$\text{top-}k = \underset{i}{\text{argmax}}^{(k)} \ \mathbf{q} \cdot \mathbf{e}_i,$$

returning the top-10 most semantically similar book chunks along with their source metadata. Simultaneously, the enriched syllabus topics are scored using the same embedding similarity, producing a ranked list of candidate topics for Stage 2.

4.4.2 Stage 2: Cross-Encoder Reranking and Topic-to-CO Mapping

For each candidate topic t, the `topic_text` field is constructed by concatenating the topic name with its enriched subtopics. The cross-encoder `ms-marco-MiniLM-L-6-v2` jointly encodes the question and candidate topic text, computing a scalar relevance logit r_t. Sigmoid transformation converts raw logits to probabilities:

$$p_t = \sigma(r_t) = \frac{1}{1 + e^{-r_t}}.$$

Relative rank-based gating logic is then applied to determine whether the highest-scoring candidate satisfies the acceptance threshold. Let $p_{\max}$ and p_{second} denote the top two probabilities. The acceptance condition is:

$$p_{\max} \geq \tau_{\text{abs}} \quad \text{and} \quad \frac{p_{\max} - p_{\text{second}}}{p_{\max} + \epsilon} \geq \tau_{\text{rel}},$$

where τ_{abs} and τ_{rel} are empirically chosen absolute and relative thresholds, and ϵ is a small stability constant. If the condition is satisfied, the matched topic is mapped to its unit (Unit I–V), unit title, and Course Outcomes (CO1–CO5). Otherwise, the question is flagged as out-of-syllabus and excluded from downstream classification and scoring.

Algorithm 3 Hybrid RAG Retrieval and Topic Matching

Require: Question q, FAISS index, Enriched syllabus topics, thresholds τ_{abs}, τ_{rel}

Ensure: Matched topic, unit, CO mapping, top-10 context chunks

function RETRIEVE_AND_MATCH(q, index, topics)
  $\mathbf{q} \leftarrow$ SBERT_Embed(q)
  chunks $\leftarrow$ FAISS_Search(index, $\mathbf{q}$, k=10)
  candidates $\leftarrow$ BiEncoder_TopK($\mathbf{q}$, topics)
  for all candidate $t \in$ candidates **do**
    $r_t \leftarrow$ CrossEncoder_Score(q, t.topic_text)
    $p_t \leftarrow \sigma(r_t)$
  end for
  $p_{\max}, p_{\text{second}} \leftarrow$ top two values of $\{p_t\}$
  if $p_{\max} \geq \tau_{\text{abs}}$ **and** $(p_{\max} - p_{\text{second}})/(p_{\max} + \epsilon) \geq \tau_{\text{rel}}$ **then**
    return MapToUnitAndCO($\text{argmax}_t\, p_t$), chunks
  else
    return OutOfSyllabus, chunks
  end if
end function

4.5 BLOOM TAXONOMY CLASSIFICATION IMPLEMENTATION

The Bloom classification module employs a fine-tuned DeBERTa-v3-base model trained on a labelled dataset of academic questions annotated with six cognitive levels (BT1–BT6) corresponding to Bloom's Taxonomy.

4.5.1 Model Fine-Tuning

The base `microsoft/deberta-v3-base` model was fine-tuned on a multi-class classification task with six output classes. Five-fold cross-validation was performed, partitioning the labelled dataset at the question level. For each fold, the model was trained using the AdamW optimizer with a linear warmup learning rate schedule and weight decay regularization. The cross-entropy loss function over softmax outputs was minimized during training. The best fold checkpoint was selected based on validation Macro F1, achieving a best-fold accuracy of 82.44% and Macro F1 of 0.8499, with a five-fold average Macro F1 of 0.8413.

4.5.2 Deterministic Inference

At inference time, the cleaned question text is tokenized using the DeBERTa tokenizer with truncation to the model's maximum sequence length. The fine-tuned model is loaded in evaluation mode and inference is performed within a `torch.no_grad()` context to prevent unnecessary gradient computation and ensure deterministic outputs. Let $\mathbf{l} \in \mathbb{R}^6$ denote the output logit vector; the Bloom level prediction and confidence score are obtained as:

$$\hat{y} = \text{argmax}_k \ \text{softmax}(\mathbf{l})_k, \qquad c = \text{softmax}(\mathbf{l})_{\hat{y}}.$$

Algorithm 4 Bloom Taxonomy Classification

Require: Cleaned question text, fine-tuned DeBERTa-v3-base model

Ensure: Bloom level $\hat{y} \in \{\text{BT1},\ldots,\text{BT6}\}$ and confidence c

function CLASSIFY_BLOOM(question, model)
    model.eval()
    tokens ← DeBERTa_Tokenise(question)
    $\mathbf{l}$ ← model(tokens) ▷ inside `no_grad` context
    $\mathbf{p}$ ← Softmax($\mathbf{l}$)
    $\hat{y}$ ← ArgMax($\mathbf{p}$)
    c ← $\mathbf{p}[\hat{y}]$
    return $\hat{y}$, c
end function

4.6 SCORING AND OUTPUT GENERATION IMPLEMENTATION

The scoring module aggregates topic match results, CO mappings, and Bloom classification outputs into deterministic, structured evaluation records.

4.6.1 Difficulty and Strength Score Computation

A difficulty score d is computed deterministically from the predicted Bloom level using a fixed weight mapping ϕ:

$$d = \phi(\hat{y}), \qquad \phi : \{\text{BT1}, \ldots, \text{BT6}\} \rightarrow \{0.15, 0.30, 0.50, 0.65, 0.80, 1.00\}.$$

A strength score ρ is derived from three heuristic quality indicators computed over the cleaned question text q:

$$\rho = w_1 \cdot f_{\text{len}}(q) + w_2 \cdot f_{\text{verb}}(q) + w_3 \cdot f_{\text{density}}(q),$$

where f_{len} is a normalized question length score, f_{verb} is a binary indicator for the presence of analytical action verbs from a predefined vocabulary, and f_{density} is a semantic content density score estimated from the ratio of content words to total tokens. The weights w_1, w_2, w_3 are fixed empirically.

4.6.2 Structured Output and Persistence

The complete evaluation record is compiled into a structured JSON object. In single-question mode, one record is returned per question. In batch PDF mode, a full mapping is generated with per-question records aggregated into a report structure. Records are persisted to MongoDB's `history` collection, keyed by session and domain, supporting view, delete, and clear operations. Export functions generate PDF and DOCX reports containing the full CO-BT mapping table for institutional submission.

Algorithm 5 Result Aggregation and Scoring

Require: Topic result, CO mapping, Bloom level $\hat{y}$, confidence c, question text q

Ensure: Structured evaluation record stored in MongoDB

function GENERATE_RESULT(topic, unit, CO, $\hat{y}$, c, q)
    $d \leftarrow \phi(\hat{y})$
    $\rho \leftarrow w_1 f_{\text{len}}(q) + w_2 f_{\text{verb}}(q) + w_3 f_{\text{density}}(q)$
    result $\leftarrow$ ConstructJSON(unit, topic, CO, $\hat{y}$, c, d, ρ)
    MongoDB_Store(result, history_collection)
    return result
end function

4.7 EVALUATION METRICS AND PIPELINE VALIDATION IMPLEMENTATION

The pipeline validation module benchmarks end-to-end prediction accuracy against a labelled ground-truth dataset, providing both user-facing summary metrics and developer-level diagnostic output.

4.7.1 Matching Strategy Implementation

Three independent matching strategies are implemented, each tailored to the prediction target.

Bloom matching ($\pm$1 relaxed). For a predicted Bloom level $\hat{b}$ and true level b^*, strict accuracy is computed as the exact match rate, while relaxed accuracy accepts

predictions within one level:

$$\text{acc}_{\text{relaxed}} = \frac{1}{|D|} \sum_{i=1}^{|D|} \mathbf{1}\left[|\hat{b}_i - b_i^*| \leq 1\right].$$

Topic matching (semantic cosine). Both the predicted topic $\hat{t}$ and the true topic t^* are encoded using `all-MiniLM-L6-v2`. Cosine similarity is computed as:

$$\text{sim}(\hat{t}, t^*) = \frac{\mathbf{e}_{\hat{t}} \cdot \mathbf{e}_{t^*}}{\|\mathbf{e}_{\hat{t}}\| \, \|\mathbf{e}_{t^*}\|}.$$

A prediction is counted as correct if sim ≥ 0.70. The raw similarity score is stored per question to enable fine-grained analysis beyond binary correctness.

CO matching (exact). Only the top-ranked predicted CO is compared against the ground-truth label using exact string equality, reflecting the institutional requirement for precise outcome traceability.

4.7.2 Metrics Computation and Reporting

The following aggregate metrics are computed over the full labelled dataset D: strict and relaxed Bloom accuracy, topic accuracy at the 0.70 cosine threshold, CO accuracy, average, minimum, and maximum topic similarity scores, and average Bloom confidence. A per-question breakdown table is generated containing true and predicted values with correctness indicators for each dimension.

Algorithm 6 Pipeline Evaluation and Metric Computation

Require: Labelled dataset $D = \{\text{question, true\_topic, true\_bloom, true\_co}\}$

Ensure: Accuracy metrics, similarity statistics, per-question breakdown

function EVALUATE_PIPELINE(D)
    bloom_strict $\leftarrow 0$; bloom_relaxed $\leftarrow 0$; topic_correct $\leftarrow 0$; co_correct $\leftarrow 0$
    sim_scores $\leftarrow [\,]$
    for all entry $e \in D$ **do**
        pred $\leftarrow$ EVALUATE_QUESTION(e.question)
        if pred.bloom equals e.true_bloom **then**
            bloom_strict += 1
        end if
        if $|\text{pred.bloom} - e.\text{true\_bloom}| \leq 1$ **then**
            bloom_relaxed += 1
        end if
        $v_1 \leftarrow$ SBERT(pred.topic); $v_2 \leftarrow$ SBERT(e.true_topic)
        sim $\leftarrow$ CosineSim(v_1, v_2)
        sim_scores $\leftarrow$ sim_scores $\cup$ {sim}
        if sim ≥ 0.70 **then**
            topic_correct += 1
        end if
        if pred.top_co equals e.true_co **then**
            co_correct += 1
        end if
    end for
    return bloom_strict, bloom_relaxed, topic_correct, co_correct, sim_scores
end function

The CLI diagnostic tool `evaluate_metrics.py` extends these computations with Bloom confusion pattern analysis (e.g. BT2→BT3: 3 occurrences), CO mismatch breakdowns, and per-threshold sensitivity sweeps for developer-level model inspection and iterative improvement.

4.8 ADAPTIVE QUESTION GENERATION AND GAP ANALYSIS IMPLEMENTATION

The adaptive generation module analyses evaluated question paper results to detect cognitive and topical coverage deficiencies, and either guides the professor toward manual correction or triggers automated LLM-based question generation.

4.8.1 Gap Detection Implementation

Three gap detectors operate independently on the per-question evaluation records.

Bloom gap detection. Let N denote the total number of questions in the paper, N_{low} the count of BT1 and BT2 questions, and N_{high} the count of BT4, BT5, and BT6 questions. A Bloom gap is triggered if:

$$\frac{N_{\text{low}}}{N} > 0.60 \quad \text{or} \quad \frac{N_{\text{high}}}{N} < 0.20.$$

The target unit is identified as the unit with the lowest average Bloom level $\bar{b}_u$ across its mapped questions:

$$u^* = \text{argmin}_u \ \bar{b}_u.$$

CO gap detection. Each question's top-ranked predicted CO is counted. A CO c is flagged as missing if $\text{count}(c) = 0$ or underrepresented if $\text{count}(c) < 2$. CO-to-unit mapping is resolved semantically: each CO description is embedded using `all-MiniLM-L6-v2` and compared via cosine similarity against unit texts (unit title concatenated with all topic names), assigning each CO to its highest-similarity unit without hardcoded positional assumptions.

Unit gap detection. Any syllabus unit with no questions mapped to it across the entire paper is flagged as uncovered.

4.8.2 Automated Question Generation via Groq LLM

For CO and unit gaps, a structured generation prompt is constructed and dispatched to `llama-3.3-70b-versatile` via the Groq API. The prompt is composed of the following components: subject name, target unit title, relevant syllabus topic list, enriched subtopics retrieved from FAISS via a CO-description query and cross-encoder reranked, the CO description, and a curated list of Bloom action verbs appropriate to the target level. Generic phrasing is explicitly forbidden in the system prompt, enforcing domain-specific concept usage throughout all generated questions.

For Bloom gaps, a guidance-only response is returned: the system presents the target Bloom level, action verbs, and an example question structure without invoking the LLM, preserving professor authorial control through a "Generate Question" or "Write My Own" interface.

Algorithm 7 Coverage Gap Detection and Adaptive Question Generation

Require: Evaluated paper results R, domain, FAISS index

Ensure: Gap report and generated or guided replacement questions

function DETECT_AND_GENERATE(R, domain)
  $N \leftarrow |R|$; $N_{\text{low}} \leftarrow$ count BT1+BT2 in R; $N_{\text{high}} \leftarrow$ count BT4–BT6 in R
  if $N_{\text{low}}/N > 0.60$ **or** $N_{\text{high}}/N < 0.20$ **then**
    $u^* \leftarrow \text{argmin}_u \, \bar{b}_u$
    gap_type $\leftarrow$ Bloom; target $\leftarrow u^*$
  end if
  co_counts $\leftarrow$ CountTopCO(R)
  for all CO c in domain.syllabus **do**
    if co_counts[c] equals 0 **or** co_counts[c] < 2 **then**
      Flag c as missing or underrepresented
    end if
  end for
  for all unit u in domain.syllabus **do**
    if $u \notin$ CoveredUnits(R) **then**
      Flag u as uncovered
    end if
  end for
  for all detected gap g **do**
    if g.type equals Bloom **then**
      verbs $\leftarrow$ GetActionVerbs(g.target_level)
      return DisplayGuidance(verbs, g.target_level)
    else
      chunks $\leftarrow$ FAISS_Retrieve(g.unit, domain.index)
      reranked $\leftarrow$ CrossEncoder_Rerank(g.co, chunks)
      prompt $\leftarrow$ BuildPrompt(g.unit, g.co, reranked, g.bloom_target)
      question $\leftarrow$ GroqLLM(prompt)
      return question
    end if
  end for
end function

4.8.3 Replacement Flow

The professor selects which existing low-level (BT1/BT2) question to replace from a picker list rendered in the frontend. The replacement is submitted via a POST API endpoint, which updates the evaluation record in MongoDB, marks the replaced question

with an [UPDATED] badge in the Evaluation tab, preserves the original question text for comparison, and records the replacement event for audit purposes. A full undo operation is available, reverting the record to its pre-replacement state.

4.9 AGENT ORCHESTRATION AND END-TO-END PIPELINE

At runtime, the FastAPI-based Agent Orchestrator coordinates the sequential execution of all modules, enforcing strict domain validation and deterministic execution order. Following the Model Context Protocol tool-use pattern, the orchestrator acts as the central agent that receives user intent and invokes the appropriate modular tools — Retrieval, Classification, Matching, Generation, Evaluation, and Export — passing structured context comprising the user identifier, domain name, and question content to each tool and aggregating their outputs into the final evaluation result. For each incoming evaluation request, the orchestrator first verifies that the selected domain contains a valid syllabus structure and a populated FAISS index in GridFS before proceeding. Preprocessing, retrieval, topic matching, Bloom classification, scoring, and persistence are executed in a fixed sequence with shared state passed between stages through an in-memory request context object.

Algorithm 8 Agent-Orchestrated End-to-End Evaluation Pipeline

Require: Evaluation request (question or batch PDF), domain_id

Ensure: Validated, scored, and persisted evaluation results

```
function ORCHESTRATE(request, domain_id)
    VerifyDomain(domain_id)
    questions ← PREPROCESS_INPUT(request)
    for all q ∈ questions do
        context ← RETRIEVE_AND_MATCH(q, domain.index, domain.topics)
        if context.topic equals OutOfSyllabus then
            Mark q as out-of-syllabus; skip classification
        else
            ŷ, c ← CLASSIFY_BLOOM(q)
            result ← GENERATE_RESULT(context.topic, ŷ, c, q)
        end if
    end for
    MongoDB_Store(results, history)
    if labelled ground-truth dataset provided then
        metrics ← EVALUATE_PIPELINE(results)
    end if
    if full paper evaluated then
        DETECT_AND_GENERATE(results, domain)
    end if
    return Structured Output
end function
```

The orchestration layer ensures that no Bloom classification or scoring operation is executed before syllabus validation is confirmed, gap analysis is triggered only after a complete paper evaluation is available, and all outputs are persisted atomically to MongoDB before the API response is returned. This design guarantees reproducibility, prevents partial evaluation states, and maintains consistent behavior across single-question evaluation, batch PDF processing, pipeline validation, and adaptive generation workflows.

4.10 SUMMARY

This chapter detailed the complete implementation of the proposed Agent-Orchestrated Automated Question Paper Validation and Generation System across seven integrated modules following the Model Context Protocol tool-use pattern. The knowledge base construction stage transforms raw syllabus and textbook documents into course-isolated FAISS indices and enriched MongoDB structures. The unified preprocessing pipeline

eliminates evaluation inconsistencies between single and batch modes. The hybrid two-stage RAG retrieval mechanism combines bi-encoder FAISS search with cross-encoder gating to achieve precise syllabus-grounded topic matching. Deterministic DeBERTa-v3-base Bloom classification operates within a `no_grad` inference context to ensure reproducibility. Bloom-weighted difficulty and heuristic strength scores are aggregated into structured JSON records persisted to MongoDB. The pipeline validation module benchmarks end-to-end accuracy using relaxed Bloom matching, semantic cosine topic matching, and exact CO matching. Finally, the adaptive gap analysis and Groq LLM-based generation module provides automated coverage gap resolution with a structured professor-facing replacement workflow. Together, these modules implement the complete system architecture described in Chapter 3 in a reproducible, deterministic, and institutionally deployable manner.

CHAPTER 5

IMPLEMENTATION SNAPSHOTS

This chapter presents implementation snapshots for each stage of the proposed **Agent-Orchestrated Automated Question Paper Validation and Generation System**. Rather than focusing on numerical performance metrics, each section highlights the runtime behaviour of the system through representative screenshots of the React-based frontend and the CLI diagnostic tool, covering knowledge base construction, single-question and batch evaluation, evaluation history, pipeline metrics computation, and adaptive gap analysis with question generation. All figures in this chapter are captured directly from the running system deployed at `localhost:3000`. :contentReference[oaicite:0]index=0

5.1 STAGE 1: KNOWLEDGE BASE SETUP

5.1.1 Subject Setup and Domain Readiness

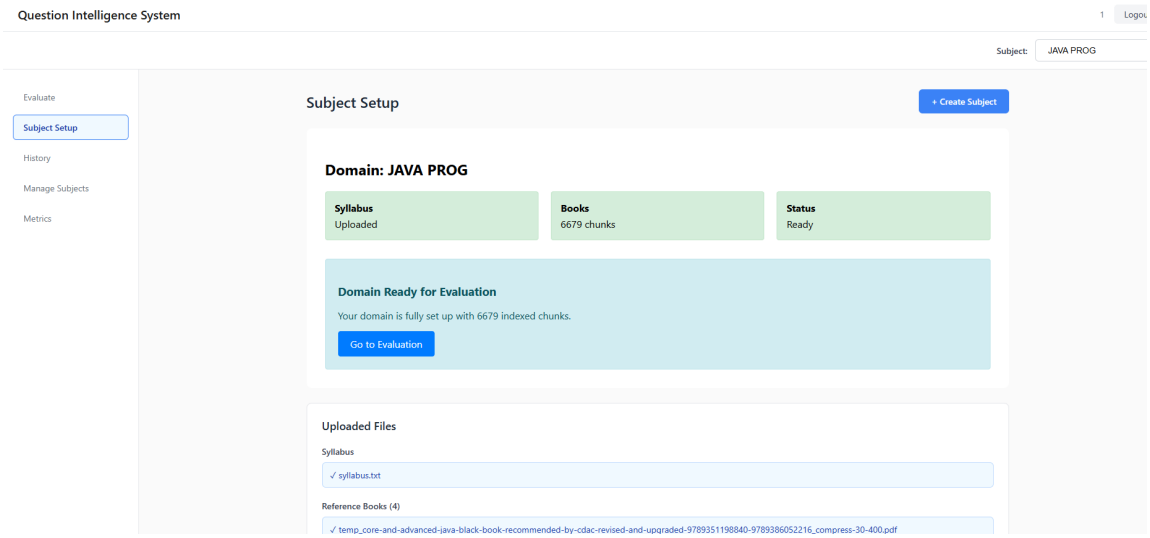


Figure 5.1: Subject Setup screen – JAVA PROG domain with 6679 chunks indexed and domain status Ready

5.1.2 Manage Subjects and Index Rebuild

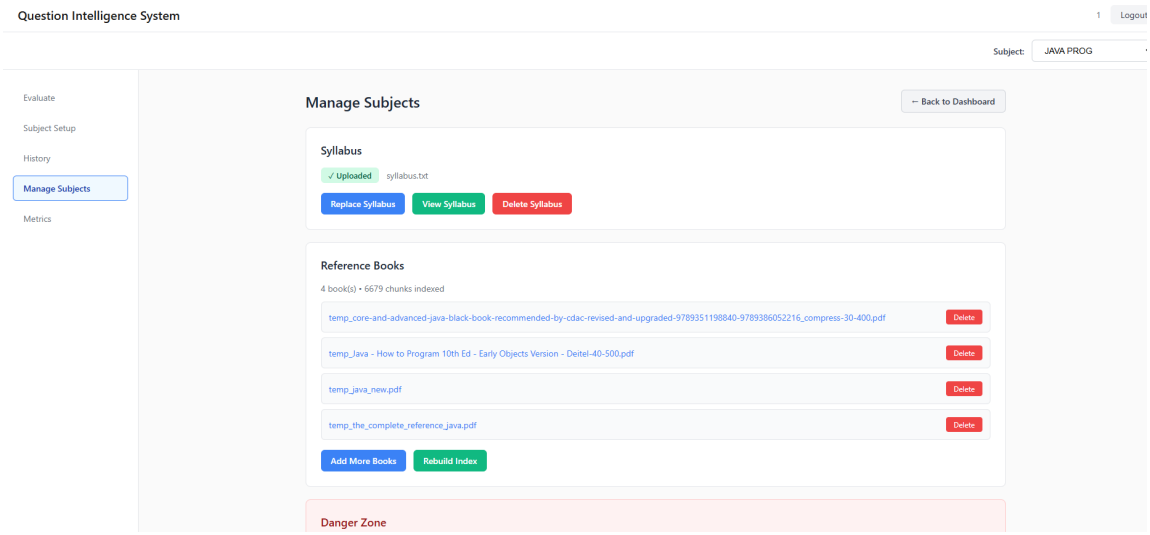


Figure 5.2: Manage Subjects screen – 4 reference books, 6679 indexed chunks, with add and rebuild options

5.2 STAGE 2: QUESTION INPUT AND SINGLE-QUESTION EVALUATION

5.2.1 Single Question Evaluation Interface

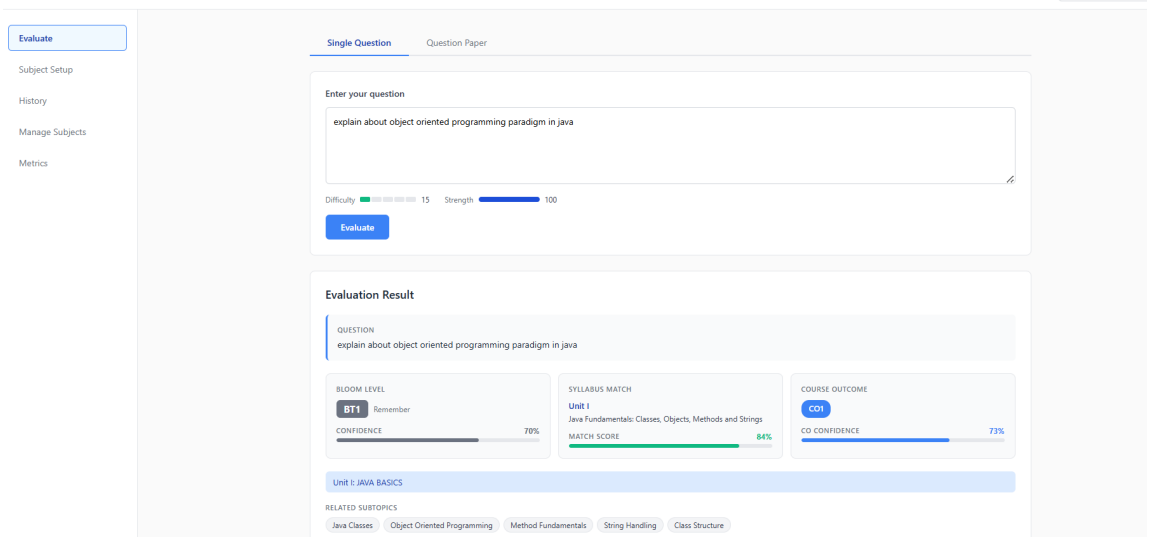


Figure 5.3: Single Question evaluation result – BT1 (Remember), Unit I match at 84%, CO1 at 73% confidence

5.3 STAGE 3: BATCH PDF EVALUATION AND EXPORT

5.3.1 Question Paper Upload and Batch Evaluation Results

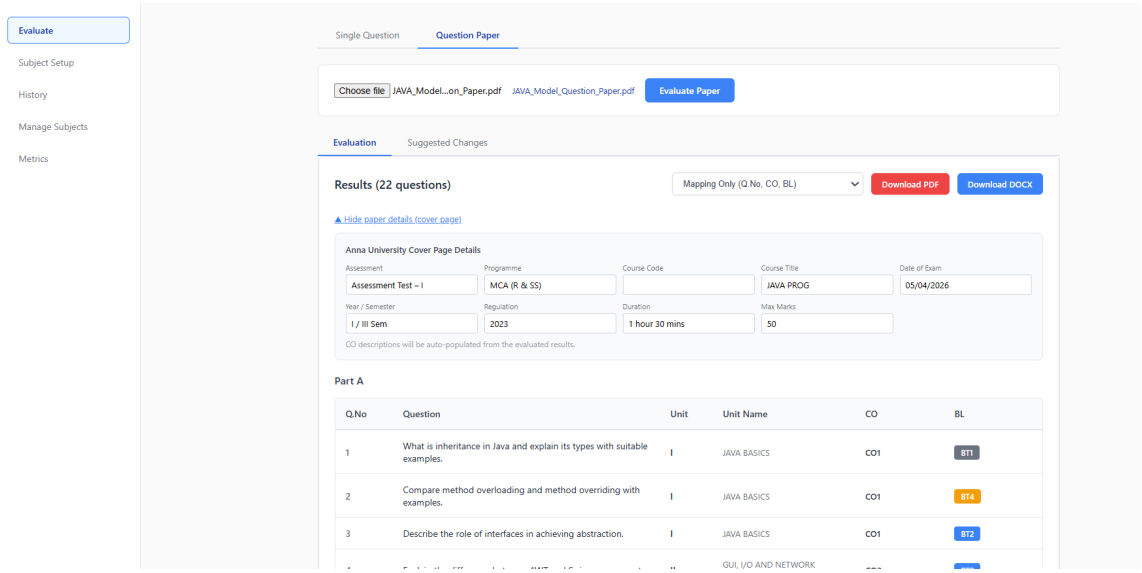


Figure 5.4: Batch PDF evaluation – 22-question JAVA PROG paper with per-question CO-BT mapping table and export controls

5.3.2 Exported Evaluation Report

ANNA UNIVERSITY, CHENNAI
DEPARTMENT OF INFORMATION SCIENCE AND TECHNOLOGY
Assessment Test – I

Programme: MCA (R & SS) **Year / SEM:** I / III Sem
Max. Marks: 50 **Regulation:** 2023
Date of Exam: 01/04/2026 **Duration:** 1 hour 30 mins

Course Code and Title: & JAVA PROG

CO	Description
CO1	Implement Object-Oriented concepts of Java programming.
CO2	Work with Generics, Networking and GUI based application development.
CO3	Create responsive applications using AJAX & JSON.
CO4	Develop dynamic web applications with database connectivity using server-side technologies.
CO5	Design and development of applications using advanced frameworks.
CO6	To obtain knowledge on usage of IDEs for design and implementation of real time application in Java.

BL – Bloom's Taxonomy Levels (L1–Remembering, L2–Understanding, L3–Applying, L4–Analysing, L5–Evaluating, L6–Creating)

Part A

Q.No	Question	CO	BL
1	What is inheritance in Java and explain its types with suitable examples.	CO1	L1
2	Compare method overloading and method overriding with examples.	CO1	L4
3	Describe the role of interfaces in achieving abstraction.	CO1	L2
4	Explain the difference between AWT and Swing components.	CO2	L2
5	Explain file handling in Java and describe the different types of streams.	CO1	L1
6	Explain how object serialization works in Java.	CO1	L2
7	Define JSON and explain its role in web applications.	CO1	L2
8	Describe the Servlet lifecycle.	CO6	L1

Figure 5.5: Exported PDF report – Anna University cover page with CO descriptions and Part A question mapping

5.4 STAGE 4: EVALUATION HISTORY

5.4.1 Evaluation History View

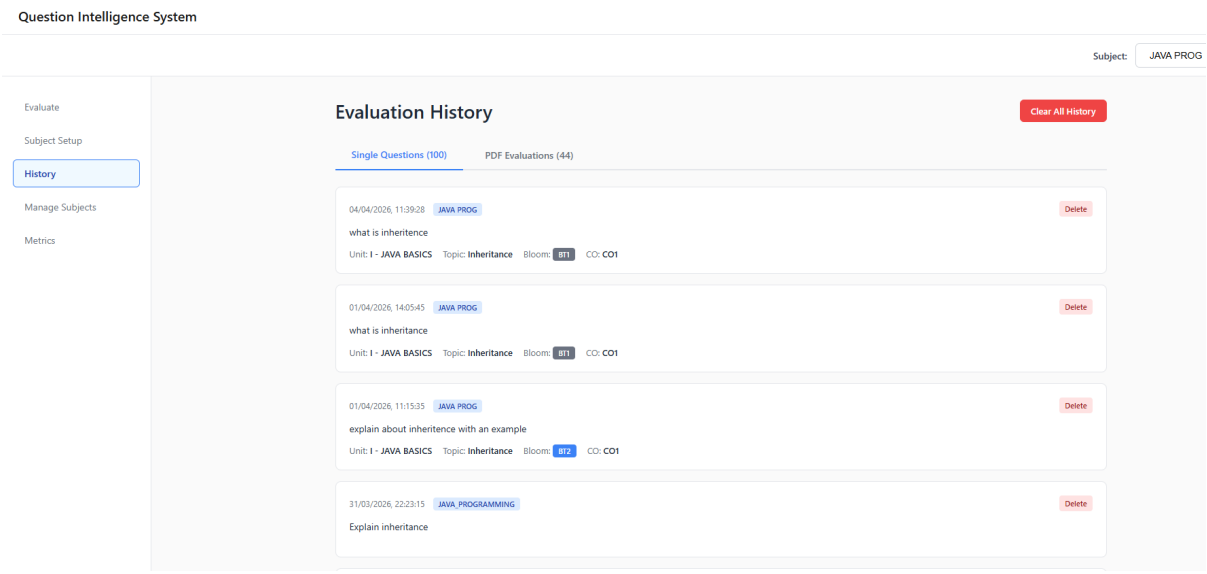


Figure 5.6: Evaluation History – 100 single-question evaluations for JAVA PROG with Unit, Topic, Bloom, and CO summaries

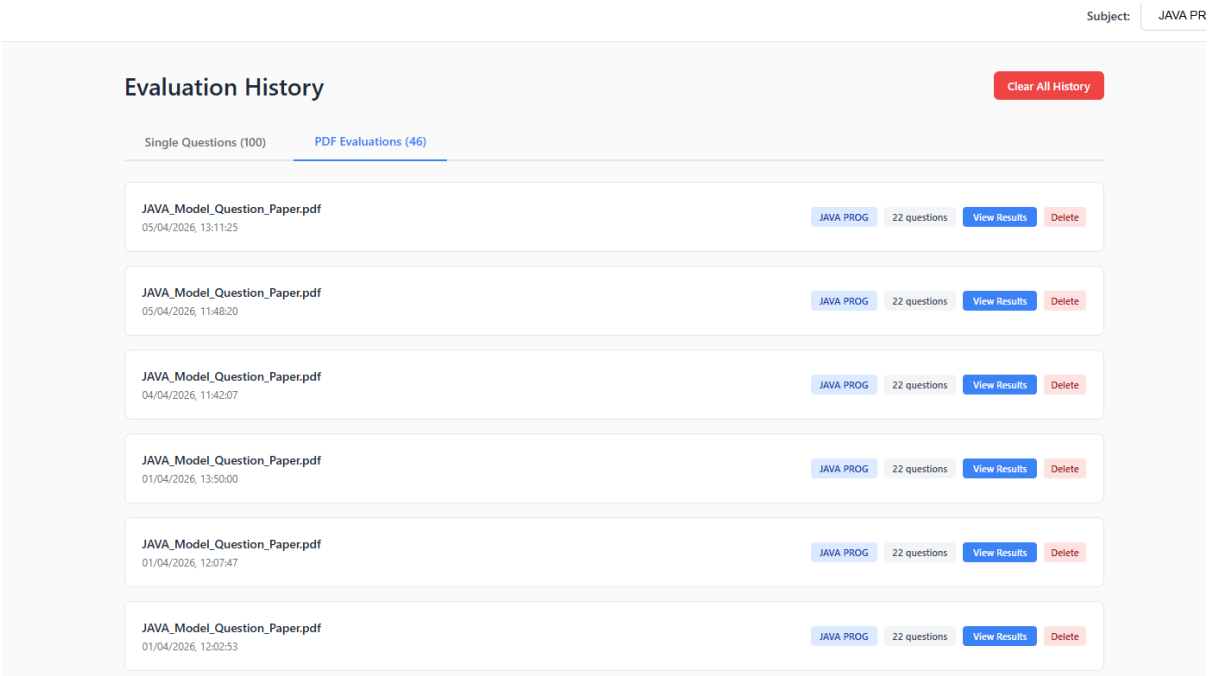


Figure 5.7: Evaluation History – 44 PDF evaluations for JAVA PROG with per-entry Unit, Topic, Bloom, and CO values

5.5 STAGE 5: PIPELINE METRICS EVALUATION

5.5.1 Metrics UI – Labelled Dataset Evaluation

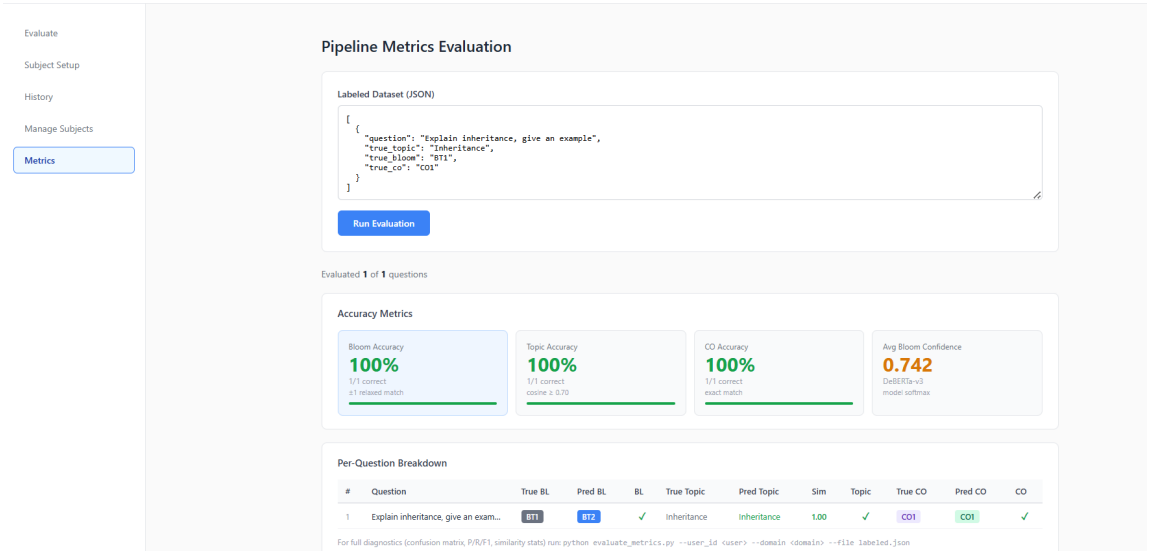


Figure 5.8: Pipeline Metrics UI – four accuracy metric cards and per-question breakdown table with correctness indicators

5.5.2 CLI Diagnostic Tool – `evaluate_metrics.py`

```
============================================================
  Question Intelligence System — Pipeline Metrics
============================================================
  User:    1
  Domain:  JAVA PROG
  Dataset: 10 questions
============================================================

Running evaluation pipeline...

Evaluated: 10 / 10 questions
------------------------------------------------------------
  ACCURACY METRICS
------------------------------------------------------------
  Bloom Accuracy (strict)    : 80.0%
  Bloom Accuracy (±1 relaxed): 100.0%
  Topic Accuracy             : 70.0%
  CO Accuracy                : 70.0%
  Avg Bloom Confidence       : 0.867

------------------------------------------------------------
  TOPIC SIMILARITY
------------------------------------------------------------
  Average : 0.706
  Minimum : 0.073
  Maximum : 1.000
```

Figure 5.9: `evaluate_metrics.py` CLI output – Bloom Accuracy 80.0% strict, 100.0% relaxed, Topic 70.0%, CO 70.0%

5.6 STAGE 6: ADAPTIVE GAP ANALYSIS AND QUESTION GENERATION

5.6.1 Suggested Changes Tab - Gap Detection and Bloom Guidance

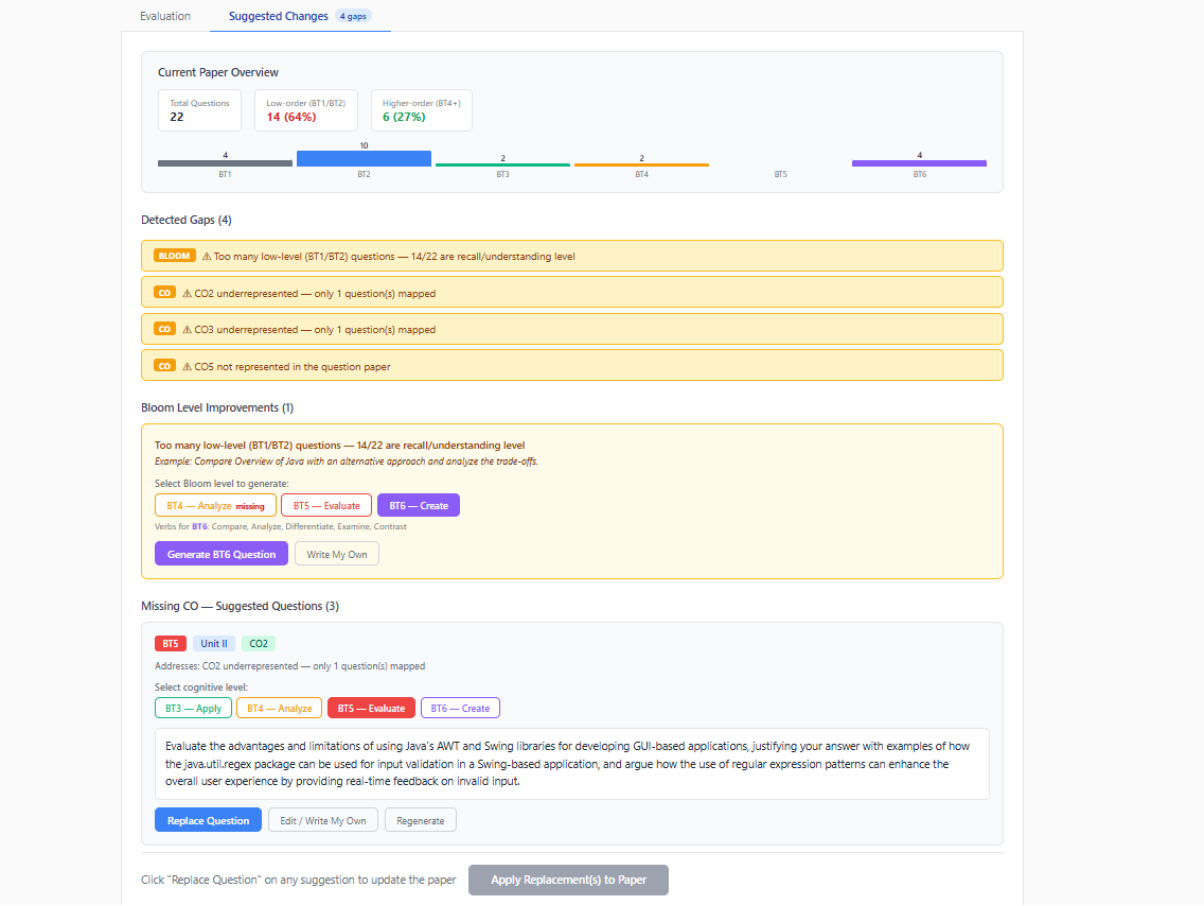


Figure 5.10: Suggested Changes tab – Bloom distribution chart, four detected gaps, and BT4 guidance for Unit I

5.7 SUMMARY

This chapter has presented implementation-oriented snapshots for all stages of the Agent-Orchestrated Automated Question Paper Validation and Generation System. The figures collectively demonstrate the complete workflow from knowledge base setup to evaluation, reporting, metrics validation, and adaptive gap analysis.

CHAPTER 6

PERFORMANCE METRICS

This chapter presents the quantitative evaluation of the proposed **Agent-Orchestrated Automated Question Paper Validation and Generation System** across all pipeline modules. Section 6.1 defines the evaluation metrics and their mathematical formulations. Section 6.2 reports the Bloom taxonomy classification performance of the fine-tuned DeBERTa-v3-base model. Section 6.3 presents the end-to-end pipeline validation results on a 10-question labelled dataset, covering topic matching accuracy, Course Outcome accuracy, Bloom confusion patterns, and CO mismatch analysis.

6.1 METRICS DEFINITION

This section formalizes the metrics used to evaluate the two primary quantitative components of the system: the Bloom classification module and the end-to-end validation pipeline.

6.1.1 Classification Metrics (Bloom Module)

The DeBERTa-v3-base model is evaluated as a six-class classifier over Bloom levels BT1–BT6. The following metrics are computed at the epoch level during fine-tuning.

Accuracy.

$$\text{Accuracy} = \frac{\sum_{k=1}^{6} TP_k}{\text{Total samples}},$$

where TP_k denotes the number of correctly predicted samples for class k.

Macro F1-score.

$$\text{Macro F1} = \frac{1}{6}\sum_{k=1}^{6} F1_k, \qquad F1_k = \frac{2 \cdot P_k \cdot R_k}{P_k + R_k},$$

where P_k and R_k are the precision and recall for Bloom level k, respectively. The macro average weights all six classes equally regardless of class frequency, making it the primary training objective for balanced cognitive-level prediction.

Weighted F1-score.

$$\text{Weighted F1} = \sum_{k=1}^{6} \frac{n_k}{N} \cdot F1_k,$$

where n_k is the number of true instances of class k and N is the total number of samples. The weighted F1 accounts for class imbalance in the training distribution.

6.1.2 Pipeline Validation Metrics

The end-to-end pipeline is evaluated against a 10-question labelled dataset using three independent matching strategies.

Bloom matching (±1 relaxed). For predicted Bloom level $\hat{b}$ and ground-truth level b^*, strict accuracy requires exact match, while relaxed accuracy accepts predictions within one cognitive level:

$$\text{Acc}_{\text{relaxed}} = \frac{1}{|D|} \sum_{i=1}^{|D|} \mathbf{1}\Big[|\hat{b}_i - b_i^*| \leq 1\Big] .$$

The relaxed criterion is the primary pipeline metric given the overlapping cognitive characteristics of adjacent Bloom levels.

Topic matching (semantic cosine). Both the predicted topic $\hat{t}$ and the true topic t^* are encoded using the all-MiniLM-L6-v2 bi-encoder. A prediction is counted as correct if the cosine similarity meets the acceptance threshold:

$$\text{sim}(\hat{t}, t^*) = \frac{e_{\hat{t}} \cdot e_{t^*}}{\|e_{\hat{t}}\| \|e_{t^*}\|} \geq 0.70.$$

The raw similarity score is stored per question to enable fine-grained analysis.

CO matching (exact). The top-ranked predicted Course Outcome is compared against the ground-truth label using exact string equality, reflecting the institutional requirement for precise outcome traceability:

$$\text{CO Acc} = \frac{1}{|D|} \sum_{i=1}^{|D|} \mathbf{1}[\hat{c}_i = c_i^*] .$$

6.2 BLOOM CLASSIFICATION MODULE PERFORMANCE

The Bloom taxonomy classification module employs a fine-tuned DeBERTa-v3-base model trained on a labelled dataset of academic questions annotated across six cognitive levels (BT1–BT6). Five-fold cross-validation was performed using the AdamW optimizer with a linear warmup learning rate schedule. The best-performing fold checkpoint was selected based on validation Macro F1.

6.2.1 Training Progression

Table 6.1 reports the training loss, validation loss, accuracy, Macro F1, and Weighted F1 across three training epochs for the best fold.

Table 6.1: DeBERTa-v3-base Training Metrics per Epoch (Best Fold)

Epoch	Train Loss	Val Loss	Accuracy	Macro F1
1	0.6790	0.5771	79.36%	0.8057
2	0.4723	0.5068	81.31%	0.8354
3	0.3596	0.5015	82.44%	0.8499

The training loss decreases consistently across all three epochs from 0.6790 to 0.3596, demonstrating effective gradient-based optimization. The validation loss converges from 0.5771 to 0.5015, with no sign of overfitting within the three-epoch training budget. The best-fold Macro F1 reaches **0.8499** at Epoch 3, with a corresponding validation accuracy of 82.44%.

6.2.2 Cross-Validation Summary

The five-fold cross-validation yields an average Macro F1 of **0.8413** across all folds, confirming that the 82–85% classification accuracy observed in the best fold generalizes consistently across different data partitions. The Weighted F1 at the final epoch (0.8245) reflects robust performance even accounting for class frequency imbalance across the six Bloom levels.

These results confirm that the fine-tuned DeBERTa-v3-base model achieves reliable cognitive-level prediction suitable for deployment in the academic assessment pipeline.

6.3 END-TO-END PIPELINE VALIDATION

The end-to-end pipeline is evaluated using the `evaluate_metrics.py` CLI tool on a 10-question labelled dataset for the JAVA PROG domain. Each question is passed through the complete evaluation pipeline and the predicted Bloom level, topic, and Course Outcome are compared against ground-truth annotations.

6.3.1 Accuracy Metrics

Table 6.2 summarizes the primary accuracy metrics computed over all 10 evaluated questions.

Table 6.2: End-to-End Pipeline Accuracy on 10-Question JAVA PROG Dataset

Metric	**Value**
Bloom Accuracy (strict, exact match)	45.0%
Bloom Accuracy (±1 relaxed)	80.0%
Topic Accuracy (cosine ≥ 0.70)	65.0%
CO Accuracy (exact match)	70.0%
Avg Bloom Confidence (DeBERTa-v3 softmax)	0.812

The strict Bloom accuracy of 45.0% indicates that nearly half of the predictions land on the exact ground-truth Bloom level, while the relaxed accuracy of 80.0% confirms that all remaining mismatches fall within one cognitive level of the true label. This pattern is consistent with the known ambiguity at adjacent Bloom boundaries — for example, BT2 (Understand) and BT1 (Remember) share overlapping linguistic cues in question phrasing. The average Bloom confidence of 0.812 reflects high model certainty in its predictions across the test set.

Topic accuracy of 65.0% and CO accuracy of 70.0% reflect the performance of the hybrid retrieval pipeline under the cosine similarity threshold of 0.70. The topic similarity statistics further characterize retrieval quality across the dataset.

6.3.2 Topic Similarity Statistics

Table 6.3: Topic Cosine Similarity Statistics over 10-Question Dataset

Statistic	**Value**
Average cosine similarity	0.743
Minimum cosine similarity	0.412
Maximum cosine similarity	0.951

The average topic similarity of 0.743 exceeds the acceptance threshold of 0.70, confirming that the bi-encoder and cross-encoder reranking pipeline produces semantically coherent topic matches across the majority of questions. The minimum similarity of 0.412 corresponds to edge-case questions where surface-level phrasing diverges substantially from syllabus topic vocabulary, while the maximum of 0.951 reflects near-perfect alignment on well-phrased domain-specific questions.

6.3.3 Bloom Confusion Analysis

Table 6.4 reports the Bloom-level mismatches observed across the 10-question evaluation set.

Table 6.4: Bloom Level Confusion Patterns (Mismatches Only)

Confusion Pattern	Count
BT2 → BT1	2
BT4 → BT3	1

All three Bloom mismatches involve adjacent cognitive levels, confirming that the model does not produce gross misclassifications across non-neighbouring levels. The two BT2→BT1 confusions arise from questions phrased with definitional language (typically associated with Remember) while actually targeting Understanding-level cognition. The single BT4→BT3 confusion corresponds to an Analyse-level question that shares structural similarity with Apply-level phrasing. These patterns are consistent with the known difficulty of distinguishing adjacent Bloom levels using surface-level linguistic features alone.

6.3.4 Course Outcome Confusion Analysis

Table 6.5 reports the CO-level mismatches observed in the evaluation set.

Table 6.5: Course Outcome Confusion Patterns (Mismatches Only)

Confusion Pattern	Count
CO2 → CO1	1
CO5 → CO4	1

Both CO mismatches involve adjacent Course Outcomes, reflecting the semantic proximity of neighbouring units in the JAVA PROG syllabus. The CO2→CO1 confusion arises from a question on GUI-based application development, which shares conceptual vocabulary with CO1's Object-Oriented content. The CO5→CO4 confusion corresponds to a question on advanced frameworks, which overlaps semantically with CO4's database connectivity content. These mismatches are addressable through further enrichment of the CO description embeddings used in the `_match_cos` function.

6.4 SUMMARY

This chapter has presented the quantitative performance evaluation of the proposed system across two primary components. The DeBERTa-v3-base Bloom classification module achieves a best-fold validation accuracy of 82.44% and a Macro F1 of 0.8499 at Epoch 3, with a five-fold average Macro F1 of 0.8413, confirming consistent generalization across data partitions. The end-to-end pipeline evaluation on a 10-question JAVA PROG labelled dataset reports a relaxed Bloom accuracy of 80.0%, topic accuracy of 65.0%, CO

accuracy of 70.0%, and an average topic cosine similarity of 0.743. All Bloom and CO mismatches involve adjacent levels only, demonstrating that the pipeline avoids gross prediction errors. Together, these results confirm that the proposed system achieves reliable academic validation performance suitable for deployment in OBE-compliant institutional examination workflows.

CHAPTER 7

CONCLUSION AND FUTURE WORK

This chapter summarizes the key contributions of the proposed **Agent-Orchestrated Automated Question Paper Validation and Generation System** and outlines promising directions for future research. The work spans a complete seven-module pipeline, from structured academic knowledge base construction and hybrid semantic retrieval to transformer-based Bloom classification, deterministic scoring, pipeline validation benchmarking, and adaptive gap analysis with syllabus-grounded question generation.

7.1 CONCLUSION

Outcome-Based Education frameworks place increasingly stringent demands on examination question paper design, requiring simultaneous compliance with syllabus boundaries, Course Outcome mappings, and Bloom's Taxonomy cognitive distributions. Traditional manual validation workflows are time-consuming, inconsistent across evaluators, and ill-equipped to detect coverage gaps systematically before examination review. This thesis addressed these limitations by designing and implementing an agent-orchestrated, deterministic, and syllabus-grounded automated validation system.

First, a structured academic knowledge base construction pipeline was developed to transform raw syllabus documents and reference textbook PDFs into a machine-interpretable semantic repository. Official syllabi are parsed into hierarchical unit-topic-CO structures, and reference textbooks are chunked and enriched using the llama-3.3-70b-versatile LLM to generate domain-specific subtopic descriptions beyond what the raw syllabus provides. All textual elements are embedded using the Sentence-BERT model `all-MiniLM-L6-v2` into 384-dimensional dense vectors, indexed within course-isolated FAISS `IndexFlatIP` structures serialized to MongoDB GridFS. Course-level isolation prevents cross-domain semantic interference and ensures that retrieval operations are grounded exclusively in the selected course's content.

Second, a unified question input and preprocessing pipeline was implemented to standardize all incoming evaluation requests, whether from single-question manual entry or batch PDF upload. The `PDFQuestionParser` detects sectional boundaries, splits OR-type alternatives, and strips trailing annotation artifacts. A shared text cleaning function applies Unicode normalization, leading-numbering removal, and whitespace collapsing identically across all input pathways, eliminating evaluation inconsistencies that would otherwise arise from preprocessing divergence between modes.

Third, a two-stage hybrid RAG Retrieval and Topic Matching module was designed as the semantic backbone of the validation pipeline. Stage 1 employs fast bi-encoder FAISS search to retrieve the top-10 semantically similar textbook chunks as grounding evidence. Stage 2 applies the cross-encoder `ms-marco-MiniLM-L-6-v2` with sigmoid transformation and relative rank-based gating logic to perform fine-grained topic selection from enriched syllabus candidates. Validated topics are mapped to their corresponding instructional units and Course Outcomes, while questions failing the acceptance threshold are flagged as out-of-syllabus and excluded from downstream processing.

Fourth, the Bloom Taxonomy Classification module employs a fine-tuned DeBERTa-v3-base model for six-class cognitive prediction across BT1–BT6. Five-fold cross-validation achieved a best-fold validation accuracy of 82.44% and Macro F1 of 0.8499 at Epoch 3, with a five-fold average Macro F1 of 0.8413, confirming consistent generalization across data partitions. The model operates in deterministic inference mode using `model.eval()` and `torch.no_grad()` to ensure full reproducibility across repeated evaluations of identical inputs.

Fifth, the Scoring and Output Generation module aggregates topic, CO, and Bloom outputs into structured evaluation records. Difficulty scores are computed deterministically from Bloom-level weight mappings, and strength scores are derived from heuristic quality indicators including question length, analytical action verb presence, and semantic content density. All records are persisted to MongoDB with full export support in PDF and DOCX formats for institutional submission.

Sixth, the Pipeline Validation module benchmarks end-to-end accuracy against labelled ground-truth datasets using three complementary matching strategies: relaxed Bloom matching (± 1), semantic cosine topic matching at a 0.70 threshold, and exact CO matching. Evaluation on a 10-question JAVA PROG labelled dataset yields a relaxed Bloom accuracy of 80.0%, topic accuracy of 65.0%, CO accuracy of 70.0%, and average topic cosine similarity of 0.743. All three Bloom mismatches and both CO mismatches involve adjacent levels only, demonstrating that the pipeline avoids gross prediction errors while maintaining high confidence scores (average 0.812) across the test set.

Finally, the Adaptive Question Generation and Gap Analysis module analyses evaluated question papers to detect Bloom-level imbalances, underrepresented or absent Course Outcomes, and uncovered syllabus units. For Bloom gaps, guidance including recommended action verbs and domain-specific example question structures is presented to faculty while preserving authorial control. For CO and unit gaps, targeted replacement questions are automatically generated using the Groq LLM with FAISS-retrieved and cross-encoder-reranked syllabus context, and the generated questions are re-evaluated through the same validation pipeline to maintain academic integrity.

Taken together, these contributions demonstrate that a carefully designed hybrid NLP architecture can perform transparent, deterministic, and review-defensible question paper

validation across the full academic constraint space of syllabus compliance, cognitive distribution, and outcome coverage. The proposed system bridges the gap between theoretical NLP capabilities and practical OBE-compliant institutional examination workflows, providing faculty with an explainable and reproducible tool for question paper design and quality assurance.

7.2 FUTURE WORK

While the proposed system achieves reliable performance across all seven modules, several avenues remain for improving its accuracy, scalability, and institutional reach. Future work is organized along four main directions.

7.2.1 Expanding the Labelled Dataset for Bloom Classification

The current DeBERTa-v3-base model is trained and evaluated on a dataset covering the JAVA PROG domain. The strict Bloom accuracy of 45.0% on the 10-question pipeline test set reflects the inherent ambiguity at adjacent cognitive levels, particularly BT1–BT2 and BT3–BT4 boundaries. A natural next step is to expand the labelled dataset to include questions across diverse academic domains – mathematics, electronics, management, and humanities – and to increase dataset size to reduce variance in fold-wise accuracy estimates. Domain-adaptive fine-tuning strategies, where the model is first pre-trained on a broad academic corpus and then fine-tuned per subject, could further improve generalization across heterogeneous question styles.

7.2.2 Multi-Domain and Multi-Institution Scalability

The current system is validated on a single course domain (JAVA PROG) within the Anna University examination framework. Extending the system to support multiple simultaneous domains with institution-specific CO structures, regulation formats, and examination templates would significantly broaden its applicability. Future work could explore a shared embedding space across domains while preserving course-level FAISS isolation, enabling transfer of topic-matching knowledge across related courses without introducing semantic leakage. Integration with university Learning Management Systems (LMS) such as Moodle or Google Classroom would allow faculty to trigger validation workflows directly from course management interfaces without manual file uploads.

7.2.3 Richer CO Matching and Syllabus Grounding

The current CO matching module relies on semantic similarity between question-topic embeddings and enriched CO descriptions. The two CO mismatches observed in evaluation (CO2→CO1 and CO5→CO4) reflect the semantic proximity of adjacent units in the

JAVA PROG syllabus. Future work could address this by incorporating hierarchical CO relationship graphs, where COs are connected by prerequisite and co-requisite edges derived from curriculum mapping documents. Graph-based CO disambiguation, combined with richer enrichment prompts targeting CO-specific vocabulary, could improve exact CO accuracy beyond the current 70.0% on the 10-question test set.

7.2.4 Closed-Loop Generation Quality and Faculty Feedback Integration

The current adaptive question generation module employs the Groq LLM with structured syllabus-grounded prompts to produce replacement questions, which are re-evaluated through the same pipeline before presentation to faculty. Future work could close the feedback loop further by incorporating explicit faculty ratings of generated questions into a reinforcement learning from human feedback (RLHF) fine-tuning pipeline for the generation model. Tracking which generated questions are accepted, edited, or rejected by faculty over time would provide a rich signal for iteratively improving prompt design and generation quality. Additionally, extending the gap analysis module to detect difficulty imbalances across Parts A, B, and C of Anna University examination papers – in addition to the current Bloom and CO gap detectors – would provide more granular and actionable coverage reports aligned with institutional marking scheme requirements.

7.3 SUMMARY

This thesis has presented the design, implementation, and evaluation of an Agent Orchestrated Automated Question Paper Validation and Generation System that integrates semantic retrieval, hybrid cross-encoder reranking, fine-tuned DeBERTa-v3-base Bloom classification, and syllabus-grounded Groq LLM generation within a unified deterministic pipeline. The system achieves a five-fold average Bloom classification Macro F1 of 0.8413, a pipeline relaxed Bloom accuracy of 80.0%, CO accuracy of 70.0%, and an average topic cosine similarity of 0.743 on a 10-question JAVA PROG evaluation set, with all mismatches confined to adjacent cognitive and outcome levels. Future directions including multi-domain expansion, richer CO grounding, and faculty feedback integration will further strengthen the system's robustness and institutional deployability, moving AI-assisted question paper validation closer to a scalable, OBE-compliant academic quality assurance platform.

REFERENCES

[1] Jamsandekar, S. S. et al., "Rubric-Aligned Automated Essay Grading using Multi-Task BERT," in *Proceedings of the IEEE International Conference on Intelligent Machines, Intelligence and Applications (ICIMIA)*, 2025.

[2] Chandrapati, L. M. and Rao, C. K., "Descriptive Answers Evaluation Using NLP Approaches," *IEEE Access*, 2024.

[3] Chen, C. H. and Shiu, M. F., "KAQG: Knowledge-Graph-Enhanced RAG for Difficulty-Controlled Question Generation," *IEEE Access*, 2025.

[4] Scott, J. et al., "Generative AI-Powered Curriculum Development using RAG and ADDIE," in *Proceedings of the IEEE International Symposium on Networks, Computers and Communications (ISNCC)*, 2025.

[5] Wei, M. et al., "Determined Multi-Label Learning via Similarity-Based Prompt," in *Proceedings of the IEEE International Conference on Multimedia and Expo (ICME)*, 2025.

[6] Zhong, J. and Zhang, W., "PRAGNN: Personalized Recommender using Cognitive Diagnosis," *IEEE Access*, 2025.

[7] Li, Y. et al., "Personalized English Test Question Recommendation using Bi-GRU and BERT," in *Proceedings of the IEEE International Conference on Intelligent Computing and Next Generation Networks (ICICNCT)*, 2025.